

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI:

HỆ THỐNG NHẬN DIỆN BỆNH DA LIỄU BẰNG HÌNH ẢNH KẾT HỢP CHATBOT HỖ TRỢ TƯ VẤN

Người hướng dẫn: ThS. MAI VĂN HÀ
Sinh viên thực hiện: CHÂU HOÀNG HUY
Số thẻ sinh viên: 102210316
Lớp: 21TCLC_KHDL2

Đà Nẵng, 06/2026

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI:

HỆ THỐNG NHẬN DIỆN BỆNH DA LIỄU BẰNG HÌNH ẢNH KẾT HỢP CHATBOT HỖ TRỢ TƯ VẤN

Người hướng dẫn: ThS. MAI VĂN HÀ
Sinh viên thực hiện: CHÂU HOÀNG HUY
Số thẻ sinh viên: 102210316
Lớp: 21TCLC_KHDL2

Đà Nẵng, 06/2026

NHẬN XÉT CỦA NGƯỜI HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Đà Nẵng, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

ThS. Mai Văn Hà

NHẬN XÉT CỦA NGƯỜI PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Đà Nẵng, ngày ... tháng ... năm 2026

TÓM TẮT

Đề tài: Hệ thống nhận diện bệnh da liễu bằng hình ảnh kết hợp chatbot hỗ trợ tư vấn

Sinh viên thực hiện: Châu Hoàng Huy

Số thẻ sinh viên: 102210316

Lớp: 21TCLC_KHDL2

Các bệnh lý da liễu có biểu hiện đa dạng, khiến nhiều người khó xác định khi nào cần thăm khám. Việc tự tìm kiếm thông tin trên Internet cũng tiềm ẩn nguy cơ tiếp nhận nội dung thiếu chính xác. Đề tài xây dựng hệ thống web hỗ trợ phân tích hình ảnh da, tra cứu thông tin qua chatbot và trao đổi trực tiếp với bác sĩ, nhằm hỗ trợ tiếp cận thông tin ban đầu, không thay thế chẩn đoán y khoa.

Hệ thống được xây dựng theo kiến trúc nhiều dịch vụ với React, Vite, Material UI ở phía giao diện; Node.js, Express, MongoDB, JWT và Socket.IO ở phía máy chủ; FastAPI cho dịch vụ AI. Chức năng nhận diện ảnh sử dụng mô hình RF-DETR để phát hiện tổn thương da và dự đoán nhóm bệnh. RF-DETR và YOLO được huấn luyện, đánh giá trên cùng tập dữ liệu để lựa chọn mô hình phù hợp.

Chatbot áp dụng phương pháp Retrieval-Augmented Generation (RAG). Cơ sở tri thức gồm 306 đoạn dữ liệu thuộc tám nhóm bệnh, được mã hóa bằng multilingual-e5-base và lưu trong ChromaDB. Hệ thống kết hợp tìm kiếm ngữ nghĩa với thông tin triệu chứng trước khi gửi ngữ cảnh tới mô hình ngôn ngữ thông qua 9router. Cơ chế phân quyền bệnh nhân và bác sĩ giúp hạn chế nội dung nhạy cảm, không kê đơn và khuyến nghị thăm khám khi cần thiết.

Kết quả là một hệ thống web tích hợp xác thực người dùng, quản lý hồ sơ, nhận diện ảnh da, lưu lịch sử nhận diện, chatbot có lịch sử hội thoại và nhắn tin thời gian thực giữa bác sĩ với bệnh nhân. Hệ thống có thể tiếp tục mở rộng dữ liệu và cải thiện chất lượng trong tương lai.

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Châu Hoàng Huy

Số thẻ sinh viên: 102210316

Lớp: 21TCLC_KHDL2 Khoa: Công nghệ Thông tin

Ngành: Công nghệ thông tin

1. *Tên đề tài đồ án*: Hệ thống nhận diện bệnh da liễu bằng hình ảnh kết hợp Chatbot hỗ trợ tư vấn
2. *Đề tài thuộc diện*: ☐ Có ký kết thỏa thuận sở hữu trí tuệ đối với kết quả thực hiện
3. *Các số liệu và dữ liệu ban đầu*: Không có
4. *Nội dung các phần thuyết minh và tính toán*:

Nội dung thuyết minh bao gồm các phần:

Mở đầu: Giới thiệu tổng quan vấn đề, mục đích và phạm vi của đề tài, hướng tiếp cận và bố cục của đồ án.

Chương 1: Cơ sở lý thuyết: Trình bày các lý thuyết, quá trình nghiên cứu và cơ sở của đồ án

Chương 2: Phân tích và thiết kế: Chương này trình bày các chức năng chính, yêu cầu của phần mềm và thiết kế dữ liệu.

Chương 3: Triển khai và đánh giá: Mô tả cách cài đặt, vận hành, kiểm thử hệ thống và kết quả đạt được.

Kết luận: Trình bày các kết quả đạt được, chỉ ra các hạn chế và đề xuất hướng phát triển.

Tài liệu tham khảo – Trình bày chi tiết các thông tin tài liệu tham khảo được sử dụng trong luận văn này.

5. *Các bản vẽ, đồ thị (ghi rõ các loại và kích thước bản vẽ)*: Không có
6. *Họ tên người hướng dẫn*: ThS. Mai Văn Hà
7. *Ngày giao nhiệm vụ đồ án*: 05/03/2025
8. *Ngày hoàn thành đồ án*: 14/06/2025

Đà Nẵng, ngày tháng năm 2026

Trưởng Bộ môn

Người hướng dẫn

LỜI NÓI ĐẦU

Trong suốt quá trình học tập tại Khoa Công nghệ Thông tin, Trường Đại học Bách khoa - Đại học Đà Nẵng, em đã nhận được sự hướng dẫn tận tình của quý thầy cô, được trang bị nền tảng kiến thức chuyên môn và rèn luyện phương pháp tư duy cần thiết cho công việc nghiên cứu, phát triển phần mềm. Đây là cơ sở quan trọng để em có thể thực hiện đồ án tốt nghiệp này.

Em xin gửi lời cảm ơn chân thành đến ThS. Mai Văn Hà, người đã trực tiếp định hướng, góp ý và hỗ trợ em trong quá trình lựa chọn giải pháp, xây dựng hệ thống và hoàn thiện nội dung đồ án. Những trao đổi chuyên môn và nhận xét của thầy giúp em nhìn nhận bài toán rõ ràng hơn, đặc biệt trong việc cân bằng giữa khả năng ứng dụng của trí tuệ nhân tạo và yêu cầu thận trọng đối với thông tin y khoa.

Em cũng xin cảm ơn gia đình, bạn bè và các anh chị đã động viên, chia sẻ kinh nghiệm và tạo điều kiện để em hoàn thành đồ án. Mặc dù đã cố gắng kiểm tra hệ thống và trình bày báo cáo một cách đầy đủ, đồ án khó tránh khỏi những thiếu sót do giới hạn về thời gian, dữ liệu và kinh nghiệm thực tế. Em kính mong nhận được những góp ý của quý thầy cô để sản phẩm có thể tiếp tục được hoàn thiện.

Em xin chân thành cảm ơn!

CAM ĐOAN

Em xin cam đoan:

1. Báo cáo đồ án tốt nghiệp đề tài: Hệ thống nhận diện bệnh da liễu kết hợp Chatbot hỗ trợ tư vấn là công trình nghiên cứu của sinh viên Châu Hoàng Huy dưới sự hướng dẫn trực tiếp của giảng viên ThS. Mai Văn Hà.
2. Em đã tự đọc nghiên cứu, dịch tài liệu và tổng hợp các kiến thức đã làm nên báo cáo này và đảm bảo không sao chép ở bất cứ đâu.
3. Những lý thuyết trong đồ án đều được sử dụng tài liệu như em đã đề cập ở phần tài liệu tham khảo đã có trong báo cáo.

Nếu có vi phạm, em xin chịu hoàn toàn trách nhiệm.

Đà Nẵng, ngày 09 tháng 06 năm 2026

Sinh viên thực hiện

Châu Hoàng Huy

MỤC LỤC

MỞ ĐẦU	1
1. Tổng quan đề tài	1
2. Mục tiêu đề tài	1
3. Đối tượng và phạm vi nghiên cứu	2
4. Phương pháp nghiên cứu	2
5. Giới hạn sử dụng và trách nhiệm y khoa	3
6. Cấu trúc báo cáo	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	4
1.1. Đặc thù bài toán nhận diện bệnh da liễu từ hình ảnh	4
1.2. Bài toán phát hiện đối tượng trong ảnh y khoa	4
1.3. Các hướng mô hình phát hiện đối tượng	5
1.4. YOLO và hướng tiếp cận CNN một giai đoạn	5
1.5. RF-DETR và hướng tiếp cận Transformer	7
1.6. So sánh lý thuyết giữa YOLO và RF-DETR	8
1.7. Bài toán chatbot tư vấn da liễu	8
1.8. Retrieval-Augmented Generation	9
1.9. Embedding, truy xuất vector và cơ sở dữ liệu vector	10
1.10. Kết chương	10
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	11
2.1. Phân tích yêu cầu	11
2.1.1. Tác nhân của hệ thống	11
2.1.2. Yêu cầu chức năng	11
2.1.3. Yêu cầu phi chức năng	12

2.2. Kiến trúc tổng thể	13
2.3. Sơ đồ use case.....	14
2.3.1. Use case tổng quan	14
2.3.2. Use case quản lý tài khoản.....	15
2.3.3. Use case nhận diện bệnh da liễu	16
2.3.4. Use case chatbot AI	17
2.3.5. Use case trò chuyện bác sĩ - bệnh nhân	18
2.4. Sơ đồ hoạt động	19
2.4.1. Đăng ký và quên mật khẩu	19
2.4.2. Nhận diện ảnh	20
2.4.3. Chatbot RAG	21
2.5. Thiết kế cơ sở dữ liệu	22
2.5.1. Collection users	22
2.5.2. Collection conversations và messages.....	23
2.5.3. Collection ai_conversations và ai_messages	24
2.5.4. Collection skin_detections.....	25
2.6. Thiết kế pipeline nhận diện bệnh da liễu	26
2.7. Thiết kế pipeline RAG.....	26
2.8. Thiết kế bảo mật và xử lý lỗi	27
2.9. Kết chương	27
CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ	28
3.1. Dữ liệu hình ảnh	28
3.2. Đánh giá mô hình YOLO và RF-DETR.....	32
3.3. Dữ liệu tri thức và quy trình xây dựng chatbot RAG	36

3.4. Môi trường triển khai.....	42
3.5. Triển khai chức năng tài khoản	42
3.6. Triển khai chức năng nhận diện	45
3.7. Triển khai chatbot AI.....	48
3.8. Triển khai chat bác sĩ - bệnh nhân.....	49
3.9. Kiểm thử chức năng.....	50
3.10. Kiểm thử an toàn chatbot.....	51
3.11. Đánh giá hệ thống.....	52
3.11.1. Kết quả đạt được.....	52
3.11.2. Hạn chế	52
3.12. Kết chương	52
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	53
1. Kết quả đạt được.....	53
2. Hạn chế	53
3. Hướng phát triển.....	53
TÀI LIỆU THAM KHẢO.....	55

DANH MỤC HÌNH VẼ

Hình 1.1: Kiến trúc mô hình YOLO.....	6
Hình 1.2: Kiến trúc mô hình RF-DETR	7
Hình 1.3: Quy trình Retrieval-Augmented Generation	9
Hình 2.1: Kiến trúc tổng thể hệ thống	13
Hình 2.2: Sơ đồ use case tổng quan.....	14
Hình 2.3: Use case quản lý tài khoản và quên mật khẩu.....	15
Hình 2.4: Use case nhận diện bệnh da liễu.....	16
Hình 2.5: Use case chatbot AI.....	17
Hình 2.6: Use case trò chuyện bác sĩ - bệnh nhân.....	18
Hình 2.7: Sơ đồ hoạt động đăng ký và quên mật khẩu bằng OTP	19
Hình 2.8: Sơ đồ hoạt động nhận diện ảnh	20
Hình 2.9: Sơ đồ hoạt động chatbot RAG.....	21
Hình 2.10: Sơ đồ dữ liệu MongoDB	25
Hình 2.11: Pipeline suy luận RF-DETR.....	26
Hình 3.1: Nguồn ảnh từ Dermatology Atlas	29
Hình 3.2: Nguồn ảnh từ DermNet	30
Hình 3.3: Tìm kiếm ảnh bổ sung trên Google Images	30
Hình 3.4: Gán nhãn vùng tổn thương trên Roboflow.....	31
Hình 3.5: Biểu đồ huấn luyện YOLO.....	33
Hình 3.6: Ma trận nhầm lẫn YOLO.....	33
Hình 3.7: Biểu đồ huấn luyện RF-DETR.....	34
Hình 3.8: Ma trận nhầm lẫn RF-DETR.....	34
Hình 3.9: Giao diện trang chủ	43

Hình 3.10: Giao diện đăng ký.....	43
Hình 3.11: Giao diện đăng nhập.....	44
Hình 3.12: Giao diện quên mật khẩu bằng OTP	44
Hình 3.13: Giao diện quản lý tài khoản.....	45
Hình 3.14: Giao diện tải ảnh nhận diện.....	46
Hình 3.15: Giao diện kết quả nhận diện.....	47
Hình 3.16: Giao diện lịch sử nhận diện.....	47
Hình 3.17: Giao diện chatbot AI	48
Hình 3.18: Giao diện danh sách hội thoại AI	49
Hình 3.19: Giao diện chat bác sĩ – bệnh nhân.....	50

DANH MỤC BẢNG BIỂU

Bảng 2.1. Yêu cầu chức năng của hệ thống	11
Bảng 2.2. Yêu cầu phi chức năng.....	12
Bảng 2.3. Thiết kế collection users	22
Bảng 2.4. Thiết kế collection conversations.....	23
Bảng 2.5. Thiết kế collection messages.....	23
Bảng 2.6. Thiết kế collection ai_conversations.....	24
Bảng 2.7. Thiết kế collection ai_messages.....	24
Bảng 2.8. Thiết kế collection skin_detections.....	25
Bảng 3.1. Thống kê tập dữ liệu hình ảnh	28
Bảng 3.2. So sánh kết quả YOLO và RF-DETR.....	32
Bảng 3.3. Thống kê dữ liệu tri thức RAG	41
Bảng 3.4. Môi trường và công nghệ triển khai.....	42
Bảng 3.5. Kịch bản kiểm thử chức năng	50
Bảng 3.6. Kịch bản kiểm thử an toàn chatbot	51

DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	Chữ viết tắt	Nội dung đầy đủ
1	AI	Artificial Intelligence - Trí tuệ nhân tạo
2	API	Application Programming Interface
3	CNN	Convolutional Neural Network
4	DETR	DEtection TRansformer
5	HTTP	Hypertext Transfer Protocol
6	JWT	JSON Web Token
7	LLM	Large Language Model
8	mAP	Mean Average Precision
9	RAG	Retrieval-Augmented Generation
10	REST	Representational State Transfer
11	RF-DETR	Roboflow Detection Transformer
12	UI	User Interface
13	UX	User Experience
14	WebSocket	Giao thức truyền thông hai chiều trên một kết nối
15	YOLO	You Only Look Once

MỞ ĐẦU

1. Tổng quan đề tài

Da là cơ quan có diện tích lớn của cơ thể và thường xuyên tiếp xúc với môi trường. Các tổn thương da có thể bắt nguồn từ nhiều nguyên nhân như phản ứng dị ứng, viêm, nhiễm trùng, rối loạn miễn dịch hoặc bệnh lây truyền. Biểu hiện bên ngoài của nhiều bệnh có nét tương đồng, trong khi hình thái tổn thương còn thay đổi theo vị trí, giai đoạn bệnh, màu da, ánh sáng và chất lượng ảnh. Vì vậy, người không có chuyên môn thường khó phân biệt và dễ đưa ra quyết định chăm sóc không phù hợp.

Sự phát triển của thị giác máy tính mở ra khả năng hỗ trợ phân tích hình ảnh da liễu. Mô hình phát hiện đối tượng không chỉ dự đoán nhãn mà còn xác định vùng tổn thương, nhờ đó kết quả trực quan hơn đối với người dùng. Tuy nhiên, một nhãn dự đoán đơn lẻ chưa đủ để giải đáp các câu hỏi về triệu chứng, nguyên nhân, chăm sóc hoặc dấu hiệu cần khám. Bởi vậy, đề tài kết hợp mô hình nhận diện hình ảnh với chatbot khai thác nguồn tri thức có kiểm soát và kênh trao đổi với bác sĩ.

Đề tài tập trung xây dựng một hệ thống web hoàn chỉnh thay vì chỉ huấn luyện mô hình riêng lẻ. Người dùng có thể tạo tài khoản và đăng nhập ngay, khôi phục mật khẩu bằng mã OTP gửi qua email, tải ảnh, xem kết quả, quản lý lịch sử, đặt câu hỏi cho chatbot và trò chuyện với bác sĩ. Kiến trúc được tách thành giao diện React, backend Node.js và dịch vụ AI FastAPI nhằm phân định rõ trách nhiệm, thuận tiện bảo trì và có thể nâng cấp từng thành phần độc lập.

2. Mục tiêu đề tài

- Xây dựng ứng dụng web có giao diện rõ ràng, hỗ trợ hai vai trò bệnh nhân và bác sĩ.
- Xây dựng luồng nhận diện bệnh da liễu từ ảnh, trả về vùng phát hiện, tên nhóm bệnh và độ tin cậy.
- Huấn luyện và so sánh YOLO với RF-DETR trên cùng phạm vi dữ liệu để lựa chọn mô hình tích hợp.
- Xây dựng chatbot RAG tiếng Việt dựa trên nguồn tri thức da liễu, có kiểm soát an toàn theo vai trò.
- Xây dựng kênh nhắn tin thời gian thực giữa bác sĩ và bệnh nhân, hỗ trợ văn bản và ảnh.

- Lưu trữ lịch sử thao tác theo người dùng và bảo vệ các API bằng cơ chế xác thực phù hợp.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng sử dụng gồm bệnh nhân hoặc người có nhu cầu tìm hiểu thông tin da liễu, và bác sĩ tham gia tư vấn qua kênh trò chuyện. Bệnh nhân được sử dụng chức năng nhận diện, chatbot ở chế độ an toàn và liên hệ bác sĩ. Bác sĩ có thể trao đổi với bệnh nhân và sử dụng chatbot ở chế độ cho phép truy xuất thêm nội dung chuyên môn, nhưng kết quả vẫn chỉ mang tính hỗ trợ.

Phạm vi nhận diện và tri thức của đồ án gồm tám nhóm: viêm da cơ địa, mề đay, vảy nến, bệnh lý niêm mạc miệng, zona thần kinh, mụn cóc, giang mai và mụn trứng cá. Đây không phải toàn bộ các bệnh da liễu và hệ thống không xử lý đầy đủ chẩn đoán phân biệt. Ảnh đầu vào được giới hạn ở định dạng JPG, JPEG hoặc PNG. Phiên bản hiện tại được triển khai dưới dạng ứng dụng web và tối ưu cho quy trình thử nghiệm.

4. Phương pháp nghiên cứu

- Khảo sát bài toán, xác định chức năng và giới hạn an toàn của hệ thống hỗ trợ thông tin y khoa.
- Thu thập, phân loại và chuẩn hóa dữ liệu ảnh theo tám nhóm bệnh; chuẩn bị dữ liệu phục vụ huấn luyện.
- Huấn luyện YOLO và RF-DETR, theo dõi các chỉ số đánh giá và lựa chọn mô hình phù hợp.
- Xây dựng nguồn tri thức, chia đoạn, gắn metadata, tạo embedding và lưu chỉ mục trong ChromaDB.
- Phân tích, thiết kế và phát triển ứng dụng theo kiến trúc frontend - backend - AI service.
- Kiểm thử theo luồng chức năng, trường hợp lỗi, phân quyền và tình huống an toàn của chatbot.
- Tổng hợp kết quả, đánh giá hạn chế và đề xuất hướng phát triển.

5. Giới hạn sử dụng và trách nhiệm y khoa

Hệ thống là công cụ hỗ trợ học tập và tham khảo. Kết quả nhận diện không phải chẩn đoán y khoa; chatbot không kê đơn, không hướng dẫn liều thuốc kê đơn và không thay thế bác sĩ. Người dùng cần đến cơ sở y tế khi tổn thương tiến triển nhanh, đau nhiều, rỉ mủ, có sốt, ảnh hưởng niêm mạc, thuộc nhóm nguy cơ hoặc xuất hiện dấu hiệu bất thường.

Giới hạn này được thể hiện cả trong nội dung báo cáo và logic hệ thống. Chatbot chỉ sử dụng ngữ cảnh truy xuất từ kho tri thức, lọc nội dung theo mức truy cập, loại bỏ hướng dẫn liều dùng đối với bệnh nhân và ưu tiên các đoạn có dấu hiệu cảnh báo. Dù vậy, các cơ chế kỹ thuật không thể loại bỏ hoàn toàn rủi ro sai lệch; việc triển khai thực tế cần có chuyên gia y tế thẩm định.

6. Cấu trúc báo cáo

Đồ án bao gồm các nội dung sau:

Mở đầu: Trình bày tổng quan về đề tài, xác định mục tiêu, phạm vi và đối tượng nghiên cứu của đề tài.

Chương 1: Cơ sở lý thuyết: Trình bày cơ sở nghiên cứu của bài toán nhận diện bệnh da liễu từ hình ảnh và chatbot tư vấn, bao gồm đặc thù dữ liệu da liễu, bài toán phát hiện đối tượng, hai hướng mô hình YOLO và RF-DETR, cùng nền tảng RAG, embedding và truy xuất vector phục vụ xây dựng chatbot có kiểm soát tri thức.

Chương 2: Phân tích và thiết kế hệ thống: Chương này đi vào quá trình khảo sát hiện trạng thực tế, phân tích và thiết kế hệ thống thông tin, xác định các tác nhân chính của hệ thống, thiết kế và xây dựng cơ sở dữ liệu.

Chương 3: Triển khai và đánh giá kết quả: Chương này sẽ đi vào quá trình triển khai hệ thống, thực hiện xây dựng mã nguồn chương trình và chạy thử nghiệm.

Kết luận: Đưa ra các đánh giá kết quả về những vấn đề đã giải quyết, những vấn đề chưa giải quyết được và các giải pháp cho để cải tiến, khắc phục những vấn đề đó.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Đặc thù bài toán nhận diện bệnh da liễu từ hình ảnh

Nhận diện bệnh da liễu từ hình ảnh không chỉ là bài toán dự đoán tên bệnh, mà còn là bài toán phân tích vùng tổn thương trên ảnh. Ảnh da liễu có độ biến thiên lớn do điều kiện ánh sáng, thiết bị chụp, khoảng cách camera, màu da, vị trí cơ thể và giai đoạn tiến triển của bệnh. Cùng một nhóm bệnh có thể xuất hiện dưới nhiều hình thái khác nhau, trong khi các nhóm bệnh khác nhau lại có biểu hiện bề mặt khá tương đồng. Vì vậy, nếu chỉ phân loại toàn bộ ảnh, hệ thống khó giải thích cho người dùng vùng nào đã tạo nên kết quả dự đoán.

Trong phạm vi đề tài, dữ liệu nghiên cứu được chắt ở tám nhóm bệnh da liễu phổ biến hoặc thường gặp trong nguồn ảnh thu thập được, gồm: viêm da cơ địa, mề đay, vảy nến, bệnh lý niêm mạc miệng, zona thần kinh, mụn cóc, giang mai và mụn trứng cá. Các nhóm này được lựa chọn vì có biểu hiện hình ảnh có thể quan sát tương đối rõ, có nguồn dữ liệu công khai để thu thập và gán nhãn, đồng thời phù hợp với mục tiêu xây dựng hệ thống hỗ trợ tham khảo. Phạm vi này không bao quát toàn bộ bệnh da liễu và không thay thế chẩn đoán của bác sĩ.

Từ đặc thù trên, hướng tiếp cận phù hợp của đề tài là phát hiện đối tượng. Mô hình không chỉ trả về nhãn bệnh mà còn xác định bounding box bao quanh vùng nghi ngờ tổn thương. Cách biểu diễn này giúp kết quả trực quan hơn, hỗ trợ người dùng hiểu vùng ảnh nào được mô hình chú ý và tạo cơ sở để so sánh chất lượng định vị giữa các mô hình.

1.2. Bài toán phát hiện đối tượng trong ảnh y khoa

Phát hiện đối tượng là bài toán xác định đồng thời vị trí và lớp của đối tượng trong ảnh. Với dữ liệu da liễu, một ảnh có thể chứa một hoặc nhiều vùng tổn thương. Mỗi dự đoán thường gồm bounding box, class và confidence. Bounding box biểu diễn vùng nghi ngờ; class là nhóm bệnh mà mô hình gán cho vùng đó; confidence thể hiện mức tin cậy của mô hình đối với dự đoán tại thời điểm suy luận.

Trong bối cảnh y khoa, confidence không được hiểu là xác suất chẩn đoán lâm sàng. Một dự đoán có confidence cao vẫn có thể sai nếu ảnh ngoài phân phối dữ liệu huấn luyện, ảnh chất lượng thấp hoặc biểu hiện bệnh không điển hình. Vì vậy, kết quả của

hệ thống cần được trình bày như thông tin hỗ trợ tham khảo, đi kèm cảnh báo rằng người dùng cần thăm khám khi triệu chứng nghiêm trọng hoặc kéo dài.

Các chỉ số thường dùng để đánh giá mô hình phát hiện đối tượng gồm precision, recall, F1-score và mAP. Precision phản ánh tỷ lệ dự đoán đúng trong số các dự đoán được đưa ra; recall phản ánh khả năng phát hiện đủ các đối tượng thật; F1-score cân bằng precision và recall. $mAP@0.5$ đánh giá dự đoán đúng khi độ chồng lấp giữa box dự đoán và box thật đạt ngưỡng IoU 0,5, còn $mAP@0.5:0.95$ đánh giá nghiêm ngặt hơn trên nhiều ngưỡng IoU, nhờ đó phản ánh tốt hơn chất lượng định vị vùng tổn thương.

1.3. Các hướng mô hình phát hiện đối tượng

Có nhiều hướng tiếp cận cho bài toán phân tích ảnh da liễu. Hướng phân loại ảnh bằng CNN chỉ dự đoán nhãn cho toàn ảnh, phù hợp khi ảnh chỉ chứa một đối tượng rõ ràng nhưng hạn chế về khả năng giải thích vị trí tổn thương. Hướng two-stage detector như Faster R-CNN thường tạo vùng đề xuất trước rồi phân loại từng vùng, có thể đạt độ chính xác tốt nhưng pipeline phức tạp và tốc độ thường chậm hơn. Hướng one-stage detector dự đoán trực tiếp box và class trong một lượt xử lý, ưu tiên tốc độ và khả năng triển khai thực tế. Gần đây, hướng Transformer-based detector mô hình hóa quan hệ toàn cục trong ảnh và xem phát hiện đối tượng như bài toán dự đoán một tập hợp kết quả.

Trong báo cáo này, YOLO và RF-DETR được nghiên cứu như hai đại diện cho hai triết lý thiết kế khác nhau. YOLO đại diện cho nhóm mô hình phát hiện một giai đoạn dựa trên đặc trưng học từ mạng CNN, trong khi RF-DETR đại diện cho hướng đưa Transformer vào phát hiện đối tượng. Chương này chỉ trình bày cơ sở lý thuyết và vai trò nghiên cứu của hai hướng mô hình; kết quả thực nghiệm và quyết định tích hợp được trình bày ở các chương sau.

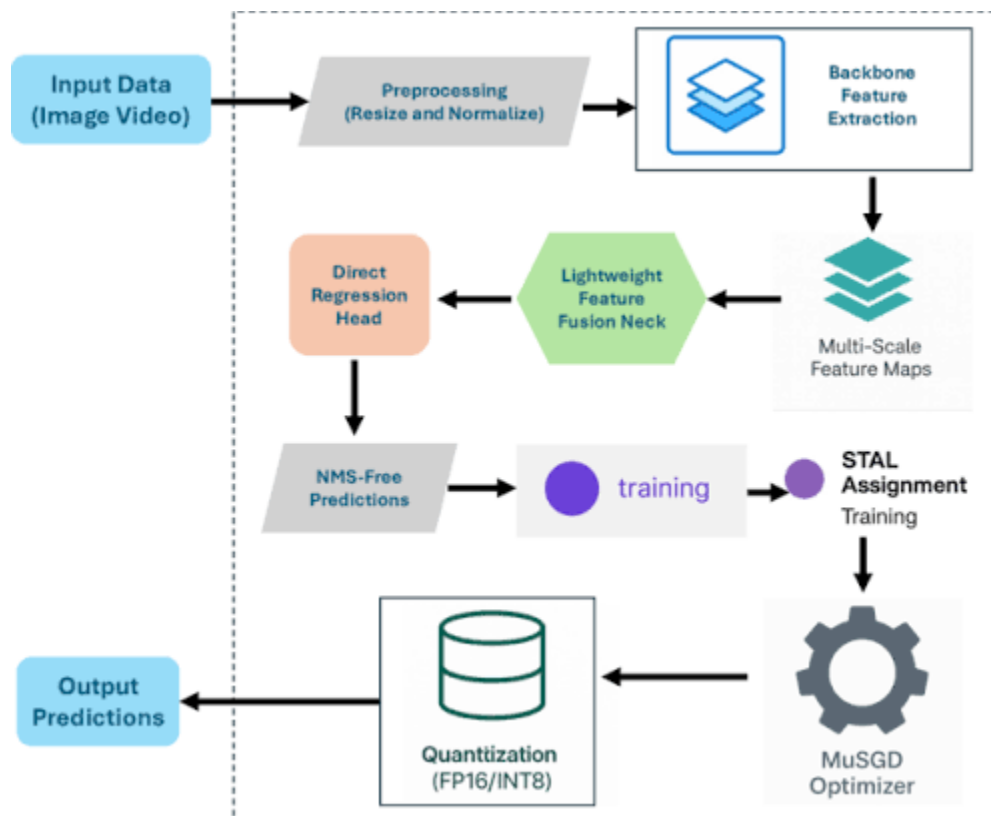
1.4. YOLO và hướng tiếp cận CNN một giai đoạn

YOLO, viết tắt của You Only Look Once, là họ mô hình phát hiện đối tượng một giai đoạn. Ý tưởng chính của YOLO là xử lý toàn bộ ảnh trong một lần truyền qua mạng để dự đoán vị trí và lớp đối tượng. Các phiên bản YOLO hiện đại thường gồm

backbone để trích xuất đặc trưng thị giác, neck để tổng hợp đặc trưng ở nhiều tỉ lệ và head để dự đoán bounding box, class cùng confidence.

Về bản chất, YOLO dựa nhiều vào khả năng học đặc trưng phân cấp của mạng CNN. Các tầng đầu thường học cạnh, màu sắc và hoa văn cục bộ; các tầng sâu hơn học cấu trúc phức tạp hơn. Với bài toán da liễu, hướng tiếp cận này có ưu điểm là tốc độ nhanh, pipeline gọn và hệ sinh thái huấn luyện, đánh giá, triển khai tương đối hoàn thiện. Đây là lý do YOLO thường được dùng làm mô hình baseline hoặc phương án phù hợp khi hệ thống cần phản hồi nhanh.

Tuy vậy, đặc trưng CNN có xu hướng phụ thuộc mạnh vào mẫu cục bộ và chất lượng dữ liệu huấn luyện. Những tổn thương nhỏ, ranh giới mờ, vùng bệnh lan tỏa hoặc các bệnh có biểu hiện gần giống nhau có thể gây khó khăn cho mô hình. Khi dữ liệu mất cân bằng, mô hình cũng có thể ưu tiên các lớp xuất hiện nhiều hơn.



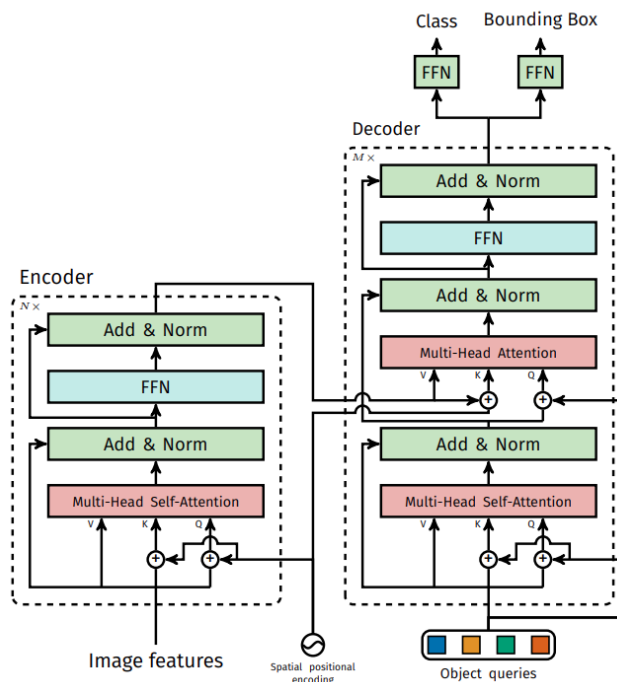
Hình 1.1: Kiến trúc mô hình YOLO

1.5. RF-DETR và hướng tiếp cận Transformer

RF-DETR được phát triển dựa trên hướng tiếp cận DETR, trong đó Transformer được đưa vào bài toán phát hiện đối tượng. Khác với các detector truyền thống thường dựa nhiều vào anchor, proposal hoặc các bước hậu xử lý thủ công, DETR xem phát hiện đối tượng như bài toán dự đoán một tập hợp kết quả. Mỗi object query học cách đại diện cho một đối tượng tiềm năng trong ảnh và dự đoán box cùng class tương ứng.

Về cơ chế tổng quát, ảnh đầu vào được backbone trích xuất đặc trưng, sau đó Transformer encoder và decoder mô hình hóa quan hệ giữa các vùng trong ảnh. Cơ chế attention giúp mô hình quan sát mối liên hệ toàn cục thay vì chỉ tập trung vào vùng cục bộ. Với ảnh da liễu, đặc điểm này có ý nghĩa vì tổn thương có thể phân bố không đều, có ranh giới không rõ hoặc cần xét tương quan giữa vùng bệnh và vùng da xung quanh.

Ưu điểm lý thuyết của RF-DETR nằm ở khả năng biểu diễn quan hệ toàn ảnh và giảm phụ thuộc vào các quy tắc hậu xử lý thủ công. Tuy nhiên, hướng Transformer thường yêu cầu tài nguyên lớn hơn, checkpoint có thể nặng hơn và thời gian huấn luyện hoặc suy luận có thể cao hơn so với các detector nhẹ. Vì vậy, RF-DETR được nghiên cứu như một hướng mô hình có tiềm năng về chất lượng định vị, nhưng cần được đánh giá thực nghiệm trên cùng dữ liệu với các mô hình khác trước khi kết luận.



Hình 1.2: Kiến trúc mô hình RF-DETR

1.6. So sánh lý thuyết giữa YOLO và RF-DETR

Xét về kiến trúc nền, YOLO thuộc nhóm detector một giai đoạn dựa trên đặc trưng CNN, còn RF-DETR thuộc nhóm detector dựa trên Transformer và cơ chế dự đoán tập hợp. YOLO thường mạnh ở tốc độ, tính đơn giản của pipeline và khả năng triển khai trong các hệ thống cần phản hồi nhanh. RF-DETR lại nhấn mạnh khả năng mô hình hóa quan hệ toàn cục và cách biểu diễn đối tượng bằng object queries.

Xét về cách học đặc trưng, YOLO khai thác đặc trưng phân cấp và cục bộ, phù hợp với nhiều bài toán phát hiện phổ thông nhưng có thể gặp khó khăn khi tổn thương nhỏ, mờ hoặc biểu hiện không điển hình. RF-DETR có khả năng xét tương quan giữa nhiều vùng ảnh, nhờ đó về mặt lý thuyết có lợi trong trường hợp hình thái tổn thương đa dạng. Đổi lại, RF-DETR thường cần tài nguyên tính toán và bộ nhớ lớn hơn. Do đó, hai hướng mô hình này bổ sung góc nhìn cho nhau: một hướng ưu tiên tốc độ và tính thực dụng, một hướng ưu tiên biểu diễn quan hệ và tiềm năng định vị.

Ở Chương 1, việc so sánh chỉ dừng ở mức lý thuyết. Báo cáo chưa sử dụng các số liệu thực nghiệm để khẳng định mô hình nào tốt hơn trong phần này. Các kết quả huấn luyện, mAP, confusion matrix và phân tích lựa chọn mô hình được đặt ở Chương 3 để bảo đảm kết luận dựa trên bằng chứng thực nghiệm.

1.7. Bài toán chatbot tư vấn da liễu

Bên cạnh nhận diện ảnh, hệ thống còn cần hỗ trợ người dùng đặt câu hỏi về thông tin da liễu. Khác với chatbot thông thường, chatbot trong phạm vi đề tài phải trả lời bằng tiếng Việt, bám vào tri thức của tám nhóm bệnh đã xác định và phân biệt nội dung phù hợp cho bệnh nhân hoặc bác sĩ. Các câu trả lời cần tránh khẳng định chẩn đoán chắc chắn, không tự ý kê đơn và phải khuyến nghị thăm khám khi có dấu hiệu nguy hiểm.

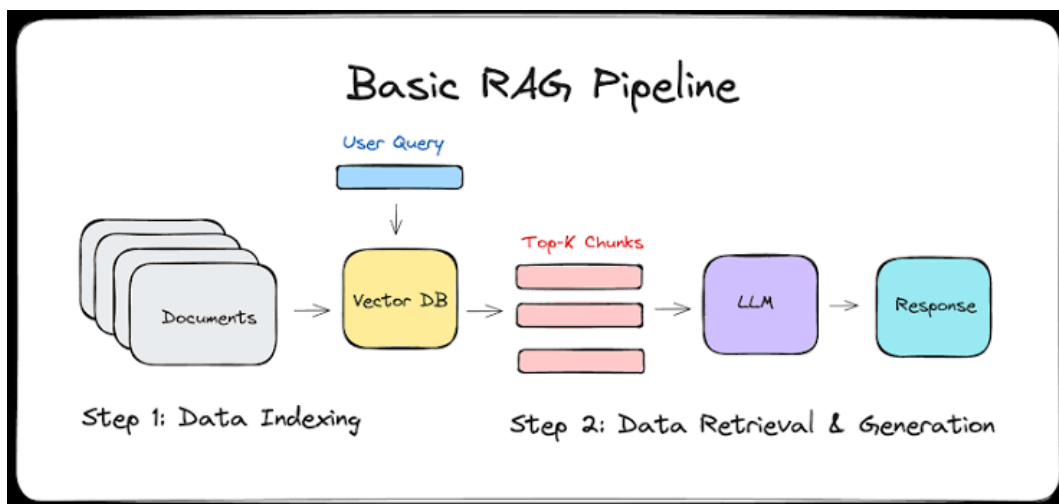
Nếu chỉ sử dụng mô hình ngôn ngữ lớn mà không có cơ chế truy xuất tri thức, hệ thống khó kiểm soát nguồn thông tin được dùng để trả lời. Mô hình có thể tạo ra nội dung nghe hợp lý nhưng không bám vào tài liệu của đề tài, đặc biệt trong các câu hỏi liên quan đến triệu chứng, điều trị hoặc cảnh báo y khoa. Vì vậy, chatbot cần một kiến trúc cho phép kết hợp khả năng sinh ngôn ngữ với nguồn tri thức đã chuẩn bị và các quy tắc an toàn.

1.8. Retrieval-Augmented Generation

Retrieval-Augmented Generation, viết tắt là RAG, là phương pháp kết hợp truy xuất tài liệu với mô hình ngôn ngữ. Thay vì để mô hình trả lời hoàn toàn từ kiến thức tham số, hệ thống trước hết tìm các đoạn tri thức liên quan đến câu hỏi, sau đó đưa các đoạn này vào prompt để mô hình sinh câu trả lời. Cách tiếp cận này phù hợp với đề tài vì kho tri thức có thể được cập nhật độc lập với mô hình ngôn ngữ và câu trả lời có cơ sở rõ hơn.

Trong bài toán tư vấn da liễu, RAG giúp kiểm soát phạm vi trả lời theo dữ liệu đã chuẩn bị. Các đoạn tri thức có thể được gắn metadata như tên bệnh, bí danh, nhóm triệu chứng, vị trí cơ thể, yếu tố khởi phát, cảnh báo nguy hiểm và mức truy cập. Khi người dùng gửi câu hỏi, hệ thống vừa dựa vào độ tương đồng ngữ nghĩa vừa sử dụng metadata để ưu tiên đoạn phù hợp. Điều này giúp chatbot không chỉ tìm văn bản gần nghĩa mà còn chọn đúng ngữ cảnh y khoa và đúng vai trò người dùng.

RAG cũng hỗ trợ yêu cầu an toàn. Với chế độ bệnh nhân, hệ thống có thể ưu tiên các đoạn `patient_safe` và hạn chế nội dung chuyên môn sâu hoặc nội dung cần bác sĩ xử lý. Với chế độ bác sĩ, hệ thống có thể truy xuất thêm các đoạn `clinician_only`. Nhờ đó, cùng một kho tri thức có thể phục vụ nhiều nhóm người dùng mà vẫn duy trì ranh giới về nội dung y khoa.



Hình 1.3: Quy trình Retrieval-Augmented Generation

1.9. Embedding, truy xuất vector và cơ sở dữ liệu vector

Embedding là cách biểu diễn văn bản thành vector số sao cho các văn bản có ý nghĩa gần nhau có vị trí gần nhau trong không gian vector. Với hệ thống hỏi đáp tiếng Việt về da liễu, embedding giúp chuyển câu hỏi của người dùng và các đoạn tri thức thành dạng có thể so sánh bằng độ tương đồng ngữ nghĩa. Mô hình multilingual-e5-base phù hợp với bối cảnh này vì hỗ trợ nhiều ngôn ngữ và có khả năng xử lý truy vấn tiếng Việt lẫn thuật ngữ tiếng Anh thường gặp trong tài liệu y khoa.

Khi xây dựng dữ liệu truy xuất, mỗi chunk không chỉ gồm nội dung chính mà còn có tiêu đề, bệnh, bí danh, mục nội dung, câu hỏi gợi ý và các tag. Việc ghép các trường này trước khi tạo embedding giúp vector phản ánh tốt hơn ngữ cảnh của đoạn tri thức. Khi truy vấn, câu hỏi được mã hóa thành vector và so sánh với các vector tài liệu bằng khoảng cách cosine để tìm các đoạn liên quan.

Cơ sở dữ liệu vector như ChromaDB được dùng để lưu trữ bền vững các embedding và metadata. Ngoài độ tương đồng ngữ nghĩa, metadata giúp hệ thống tăng khả năng kiểm soát kết quả theo bệnh, triệu chứng, vị trí tổn thương, mức cảnh báo và vai trò người dùng. Mô hình ngôn ngữ được gọi thông qua lớp dịch vụ có thể thay đổi bằng cấu hình, trong khi logic truy xuất và kiểm soát an toàn vẫn nằm trong pipeline RAG của hệ thống.

1.10. Kết chương

Chương 1 đã trình bày cơ sở lý thuyết xuất phát từ hai yêu cầu chính của đề tài: nhận diện bệnh da liễu từ hình ảnh và chatbot hỗ trợ tư vấn. Đối với nhận diện ảnh, chương đã làm rõ bài toán phát hiện đối tượng, các chỉ số đánh giá và hai hướng mô hình đại diện là YOLO dựa trên CNN một giai đoạn và RF-DETR dựa trên Transformer. Đối với chatbot, chương đã trình bày RAG, embedding và truy xuất vector như nền tảng để xây dựng hệ thống trả lời có kiểm soát tri thức.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu

2.1.1. Tác nhân của hệ thống

Hệ thống có hai tác nhân nghiệp vụ chính. Bệnh nhân là vai trò mặc định khi đăng ký, có thể nhận diện ảnh, hỏi chatbot và trò chuyện với bác sĩ. Bác sĩ có thể xem danh sách bệnh nhân đang hoạt động, tạo hoặc mở hội thoại và trả lời tin nhắn. Cả hai vai trò đều quản lý hồ sơ cá nhân và có lịch sử chatbot riêng.

2.1.2. Yêu cầu chức năng

Bảng 2.1. Yêu cầu chức năng của hệ thống

Mã	Nhóm	Mô tả yêu cầu
F01	Tài khoản	Đăng ký bằng email, mật khẩu và vai trò; tài khoản hoạt động ngay.
F02	Tài khoản	Khôi phục mật khẩu bằng OTP 6 số gửi qua email, có hết hạn và giới hạn số lần thử.
F03	Tài khoản	Đăng nhập, đăng xuất và làm mới access token.
F04	Hồ sơ	Xem và cập nhật tên hiển thị, ảnh đại diện hoặc mật khẩu.
F05	Nhận diện	Chọn/kéo thả ảnh JPG, JPEG, PNG và xem trước trước khi gửi.
F06	Nhận diện	Nhận ảnh đã khoanh vùng, danh sách bệnh và độ tin cậy.
F07	Nhận diện	Xem và xóa lịch sử nhận diện thuộc tài khoản.
F08	Chatbot	Tạo, đổi tên, xem và làm sạch hội thoại AI.
F09	Chatbot	Gửi câu hỏi và nhận trả lời dựa trên kho tri thức RAG.
F10	Chat trực tiếp	Tạo hội thoại giữa một bác sĩ và một bệnh nhân.
F11	Chat trực tiếp	Gửi và nhận tin nhắn văn bản/hình ảnh theo thời gian thực.
F12	Chat trực tiếp	Xem lịch sử và xóa nội dung hội thoại.

2.1.3. Yêu cầu phi chức năng

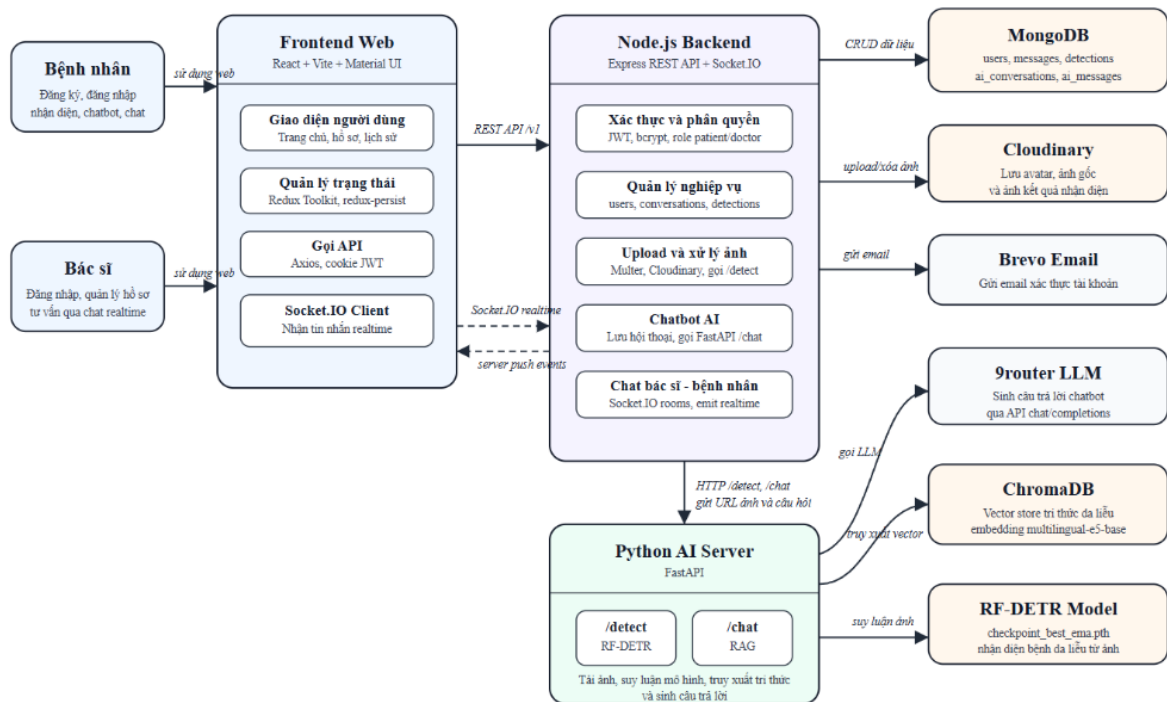
Bảng 2.2. Yêu cầu phi chức năng

Mã	Thuộc tính	Yêu cầu
NF01	Bảo mật	API nghiệp vụ phải xác thực; dữ liệu theo người dùng phải kiểm tra quyền sở hữu.
NF02	An toàn	Không trình bày kết quả AI như chẩn đoán; chatbot không kê đơn cho người dùng.
NF03	Khả dụng	Giao diện phản hồi trạng thái tải, lỗi và trường hợp không có dữ liệu.
NF04	Hiệu năng	Các tác vụ web thông thường phản hồi nhanh; suy luận AI có thông báo đang xử lý.
NF05	Mở rộng	Frontend, backend và AI service có thể triển khai, nâng cấp độc lập.
NF06	Tương thích	Giao diện hoạt động trên trình duyệt hiện đại và thích nghi kích thước màn hình.
NF07	Bảo trì	Mã nguồn phân lớp, cấu hình nhạy cảm đặt trong biến môi trường.
NF08	Toàn vẹn	Ảnh lịch sử được xóa đồng bộ trên Cloudinary khi bản ghi bị xóa.

2.2. Kiến trúc tổng thể

Trình duyệt chạy ứng dụng React và gửi REST request tới backend Express. Backend xác thực người dùng, xử lý nghiệp vụ, truy cập MongoDB, gửi mã OTP đặt lại mật khẩu qua Brevo và quản lý ảnh trên Cloudinary. Đối với nhận diện và chatbot, backend gọi Python Server bằng HTTP nội bộ. Socket.IO chạy chung HTTP server với Express để phục vụ chat trực tiếp.

Luồng nhận diện sử dụng Cloudinary như cầu nối ảnh. Backend upload ảnh gốc, gửi URL cho Python Server, nhận ảnh JPEG kết quả rồi upload lại. Cách này tránh truyền file lớn qua nhiều lớp trong cùng một request và tạo URL ổn định để lưu lịch sử. Luồng chatbot chỉ truyền JSON gồm câu hỏi và mode; dữ liệu vector được quản lý riêng trong Python Server.

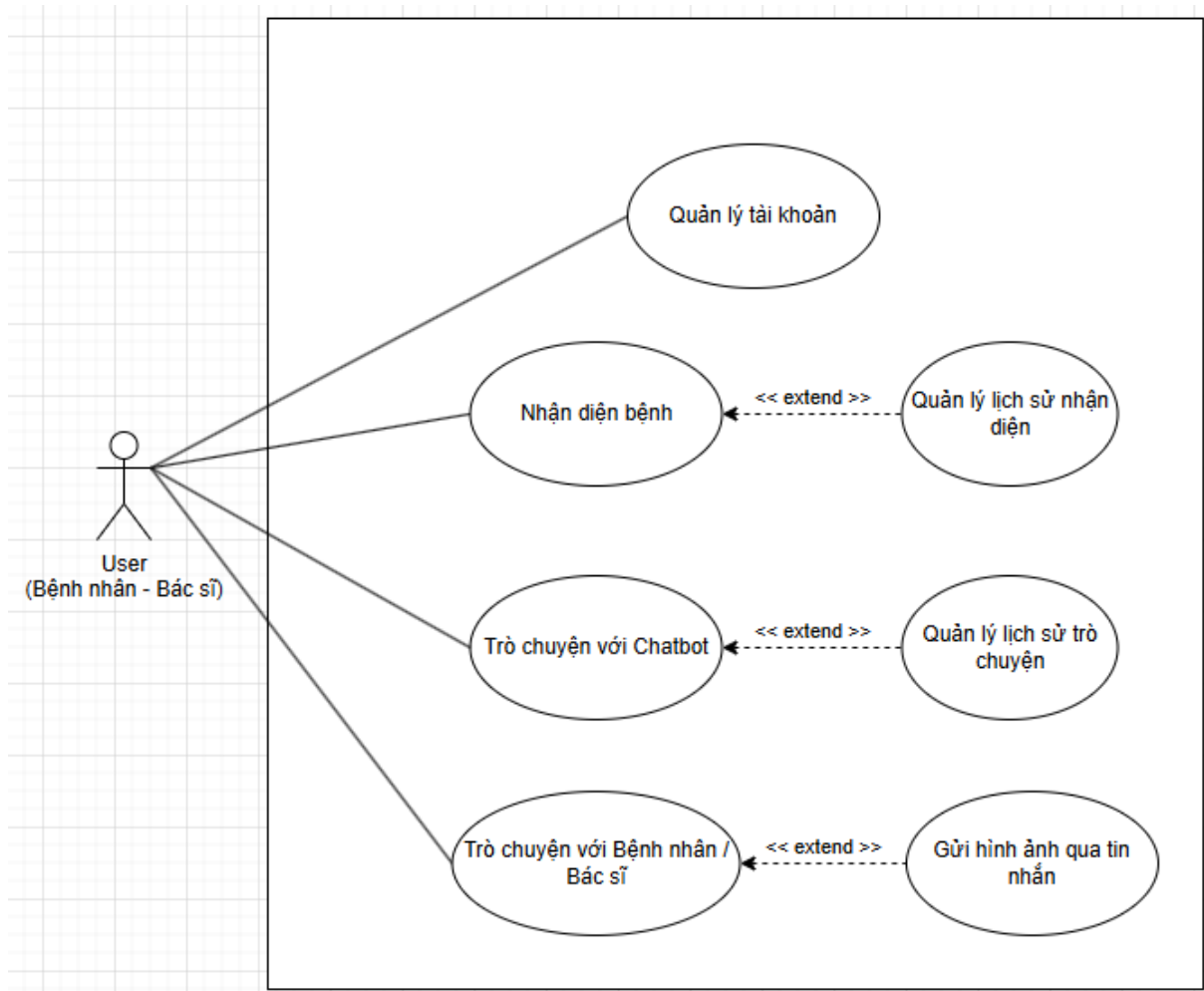


Hình 2.1: Kiến trúc tổng thể hệ thống

2.3. Sơ đồ use case

2.3.1. Use case tổng quan

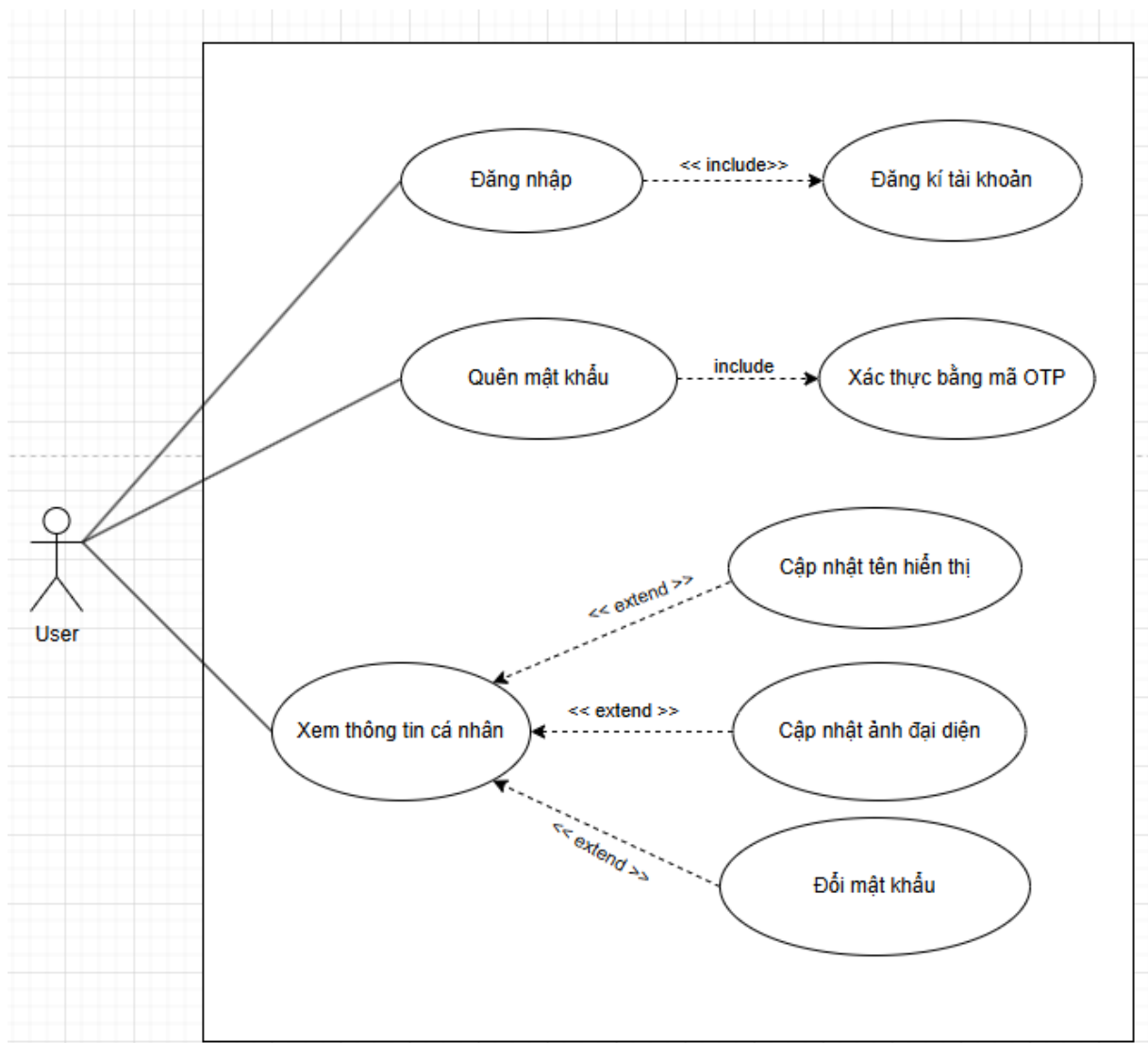
Use case tổng quan thể hiện hai tác nhân cùng sử dụng nhóm chức năng tài khoản và chatbot. Bệnh nhân nổi bật ở chức năng nhận diện ảnh và liên hệ bác sĩ. Bác sĩ nổi bật ở việc tiếp nhận danh sách bệnh nhân và trả lời hội thoại. Người dùng có thể đăng ký, đăng nhập và khôi phục mật khẩu bằng OTP; các use case nghiệp vụ yêu cầu người dùng đã đăng nhập.



Hình 2.2: Sơ đồ use case tổng quan

2.3.2. Use case quản lý tài khoản

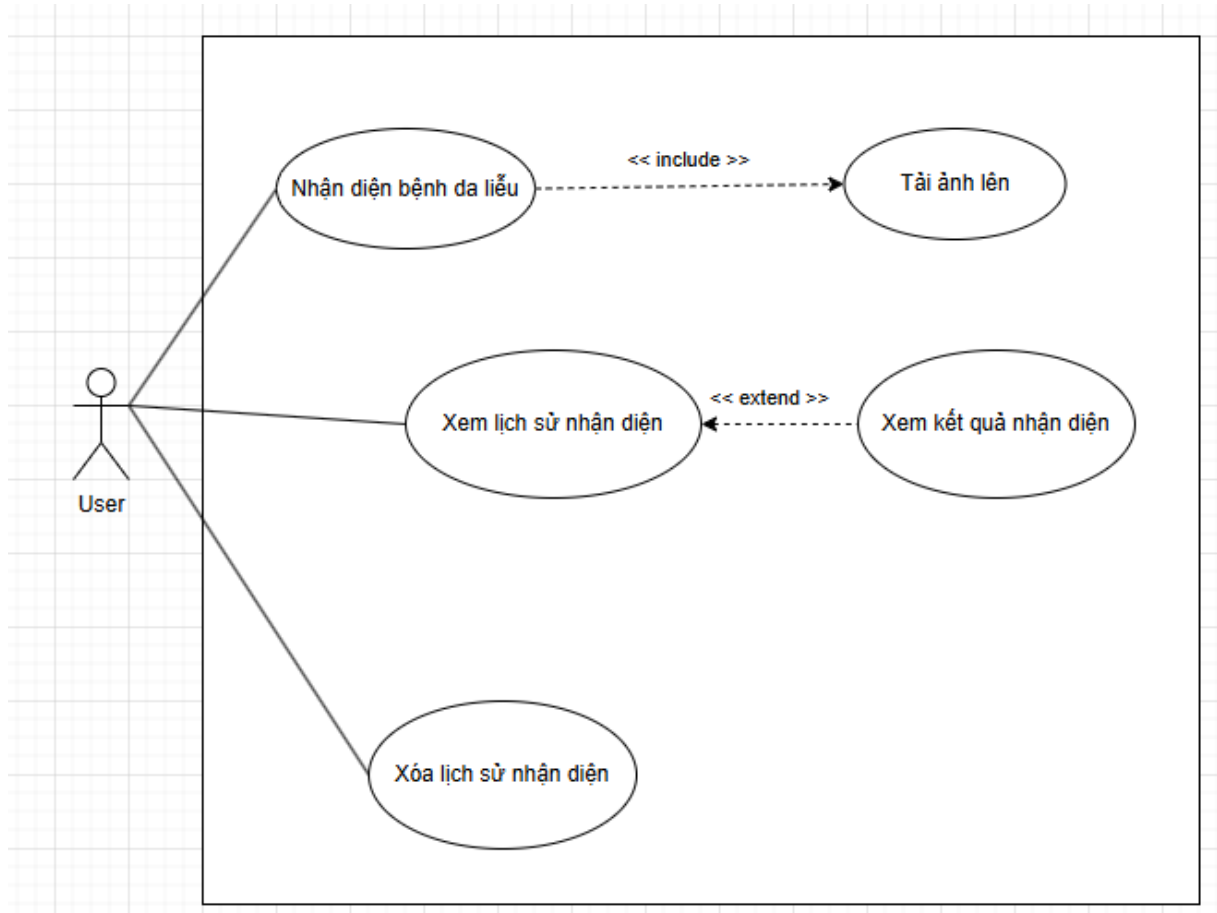
Người dùng đăng ký và có thể đăng nhập ngay mà không cần xác thực email. Khi quên mật khẩu, người dùng nhập email, nhận OTP sáu chữ số, xác nhận OTP và tạo mật khẩu mới mà không cần cung cấp mật khẩu cũ. Khi đăng nhập thành công, người dùng có thể xem hồ sơ, thay đổi tên hiển thị, ảnh đại diện hoặc mật khẩu. Đăng xuất làm mất hiệu lực phiên hiện tại ở phía trình duyệt; refresh token hỗ trợ cấp lại access token khi phiên còn hợp lệ.



Hình 2.3: Use case quản lý tài khoản và quên mật khẩu

2.3.3. Use case nhận diện bệnh da liễu

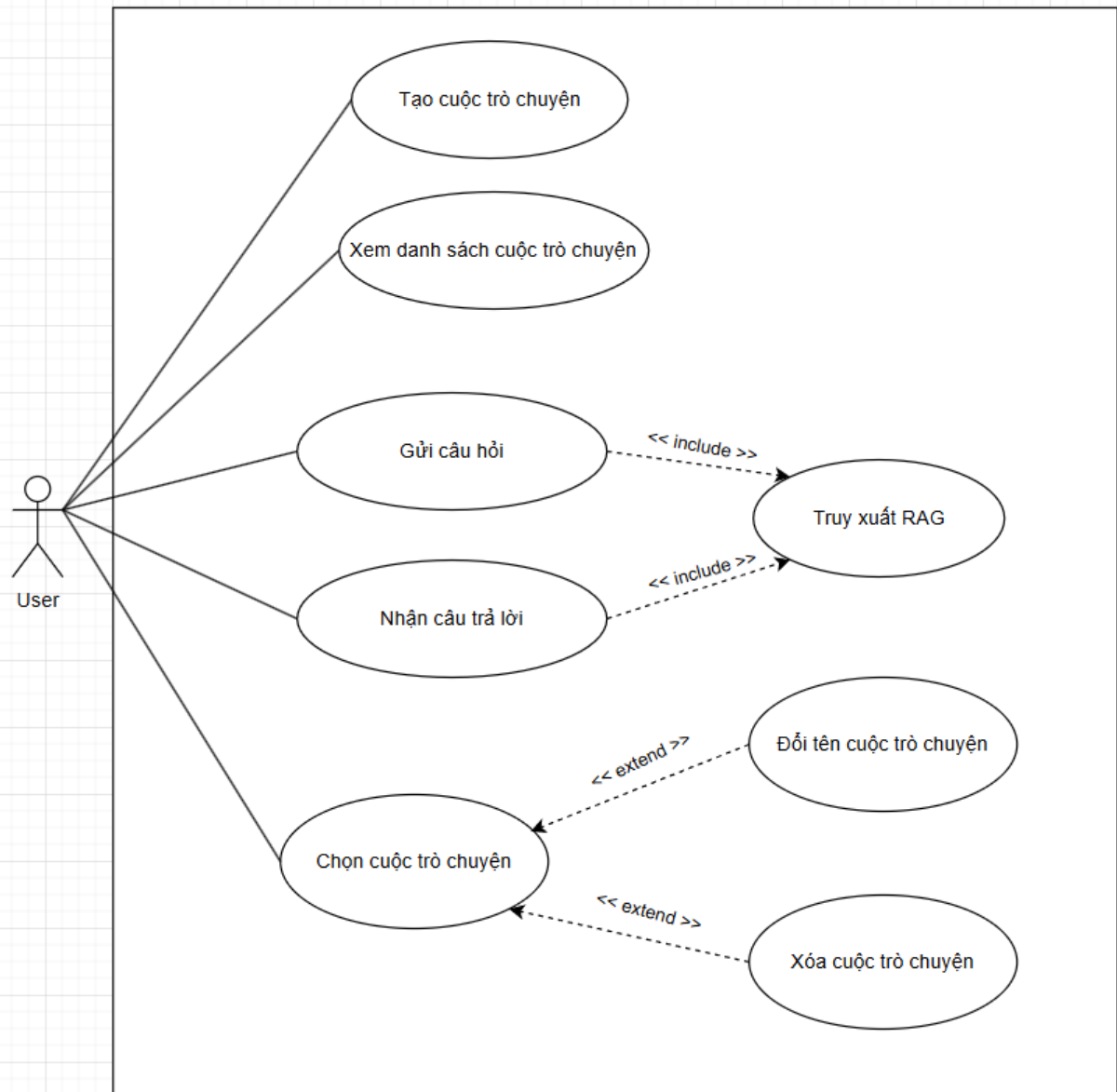
Bệnh nhân chọn ảnh, kiểm tra preview và yêu cầu nhận diện. Hệ thống kiểm tra định dạng, upload, suy luận và hiển thị kết quả. Người dùng có thể mở ảnh kết quả, phóng to, xem danh sách dự đoán, xem lịch sử và xóa một mục. Use case xóa bao gồm xóa ảnh gốc, ảnh kết quả và bản ghi.



Hình 2.4: Use case nhận diện bệnh da liễu

2.3.4. Use case chatbot AI

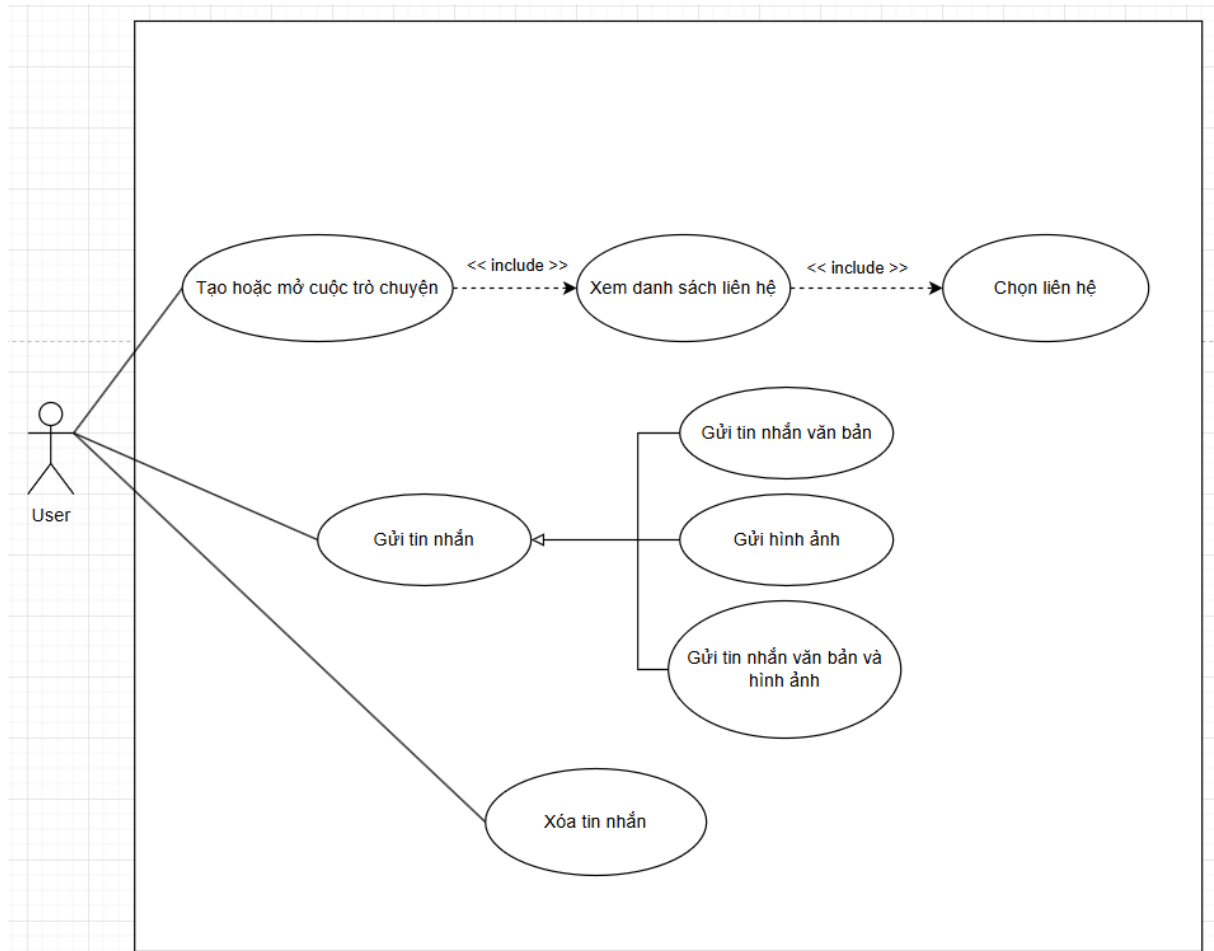
Người dùng tạo cuộc trò chuyện AI, gửi câu hỏi, xem câu trả lời và quản lý danh sách hội thoại. Mỗi hội thoại có tiêu đề và lịch sử riêng. Backend tự xác định mode patient hoặc doctor từ vai trò tài khoản; client không được tự nâng quyền truy xuất.



Hình 2.5: Use case chatbot AI

2.3.5. Use case trò chuyện bác sĩ - bệnh nhân

Người dùng xem danh sách đối tượng có vai trò đối ứng. Khi chọn một người, hệ thống tạo mới hoặc lấy hội thoại đã tồn tại. Hai bên gửi tin nhắn văn bản hoặc hình ảnh, nhận sự kiện realtime và có thể làm sạch lịch sử. Hệ thống không cho phép tạo hội thoại với chính mình hoặc giữa hai tài khoản cùng vai trò.

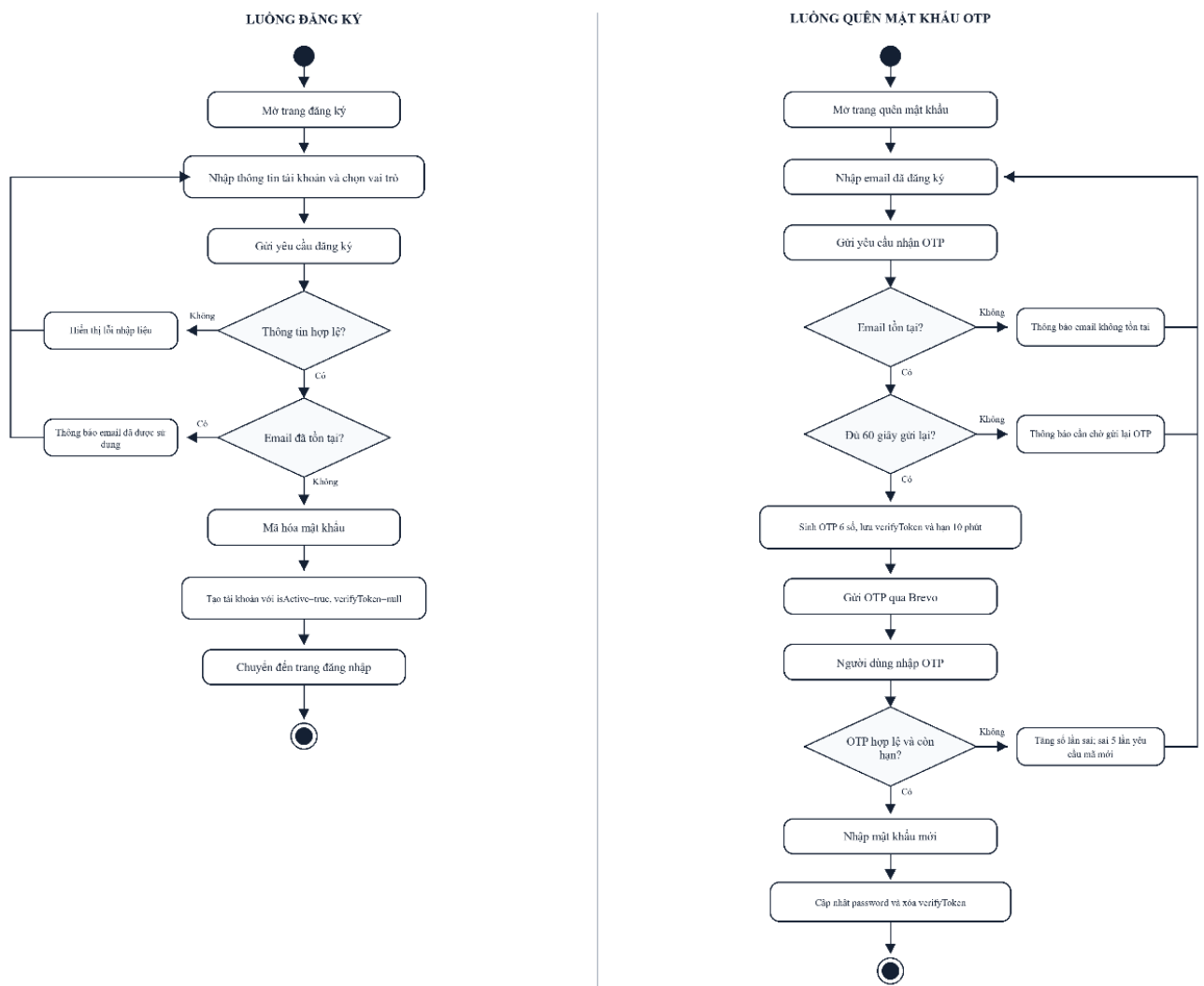


Hình 2.6: Use case trò chuyện bác sĩ - bệnh nhân

2.4. Sơ đồ hoạt động

2.4.1. Đăng ký và quên mật khẩu

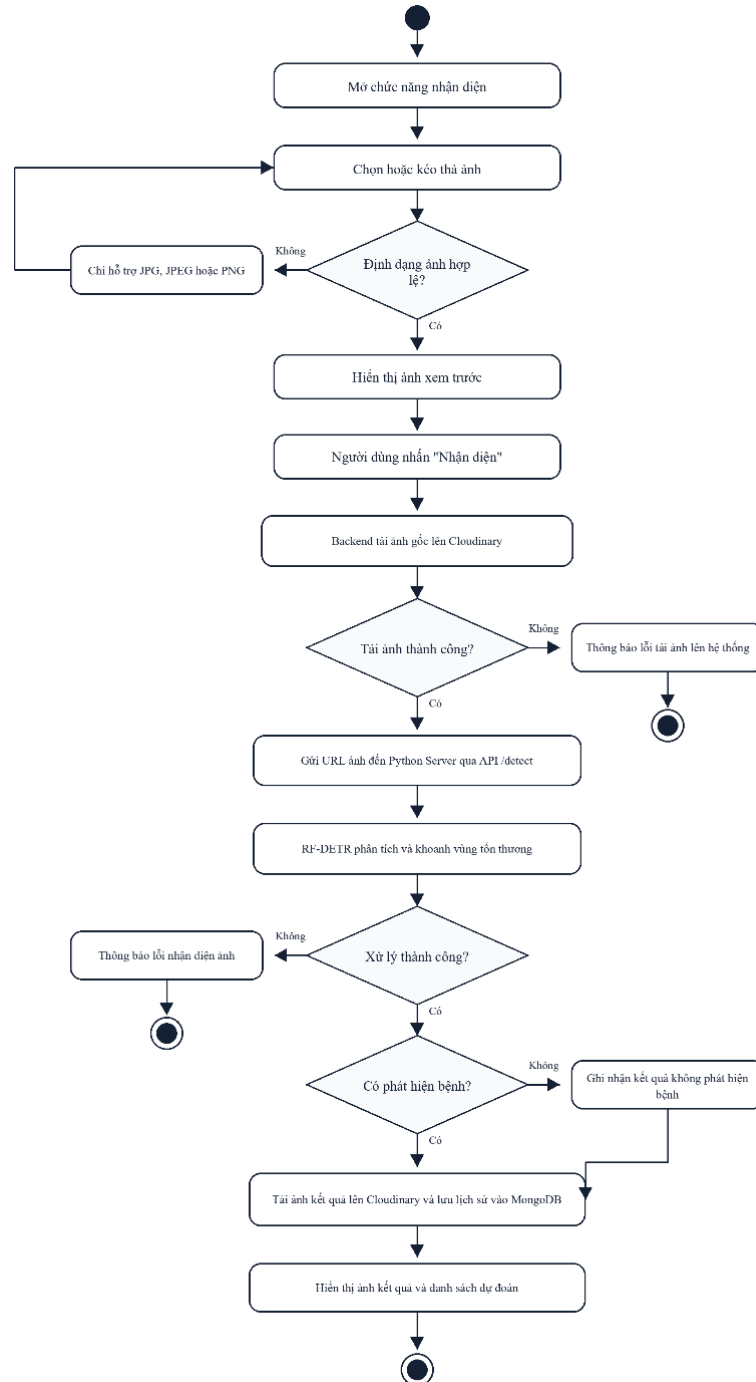
Khi đăng ký, frontend và backend kiểm tra dữ liệu, backend kiểm tra email trùng, băm mật khẩu và tạo tài khoản với `isActive=true` để người dùng có thể đăng nhập ngay. Khi quên mật khẩu, người dùng nhập email; backend sinh OTP sáu chữ số bằng bộ sinh số ngẫu nhiên an toàn, lưu OTP cùng thời điểm gửi, thời hạn 10 phút và số lần thử, sau đó gửi mã qua Brevo. Frontend xác nhận OTP trước khi cho nhập mật khẩu mới. Backend kiểm tra lại OTP, băm mật khẩu mới và đặt `verifyToken` cùng các trường OTP về null hoặc 0 khi hoàn tất.



Hình 2.7: Sơ đồ hoạt động đăng ký và quên mật khẩu bằng OTP

2.4.2. Nhận diện ảnh

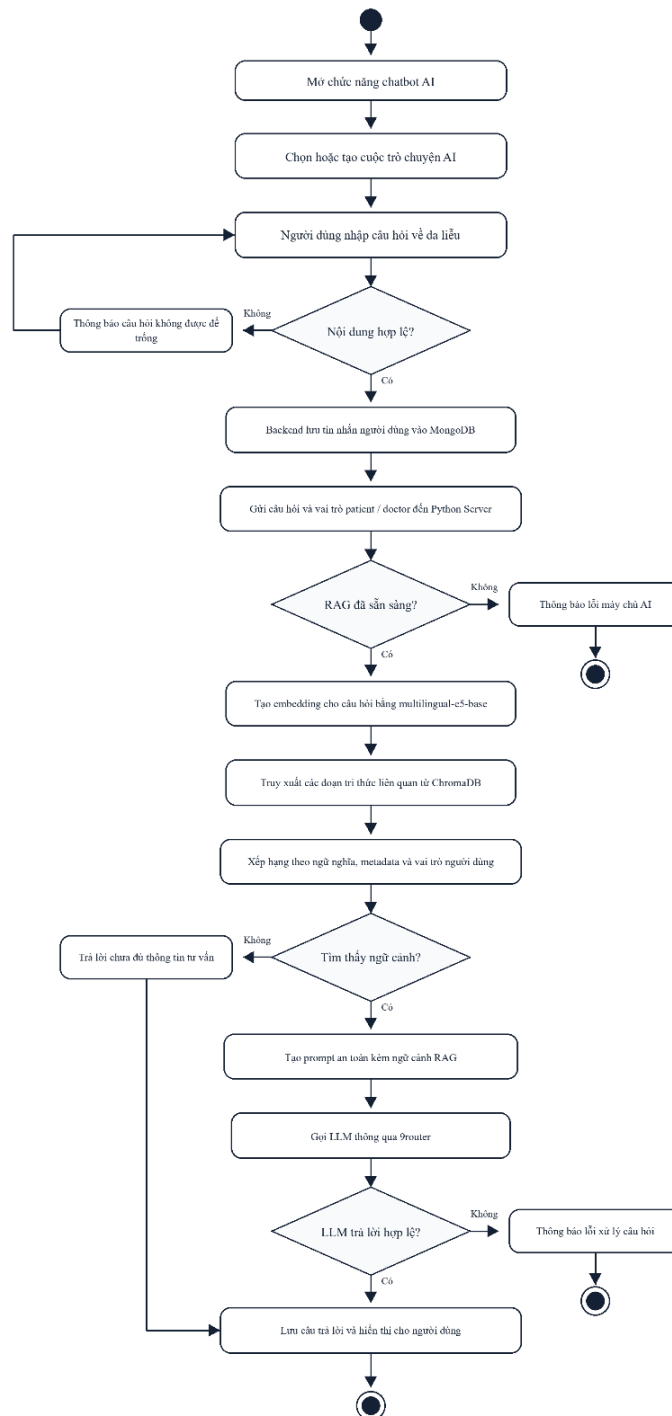
Frontend kiểm tra loại file và hiển thị preview. Khi xác nhận, backend nhận buffer, upload ảnh gốc và gọi /detect. Python Server kiểm tra URL, tải ảnh, chạy RF-DETR, vẽ kết quả và trả JPEG. Backend upload ảnh kết quả, lưu bản ghi và trả dữ liệu cho giao diện. Nếu một bước lỗi, hệ thống trả thông báo phù hợp và không tạo lịch sử không hoàn chỉnh.



Hình 2.8: Sơ đồ hoạt động nhận diện ảnh

2.4.3. Chatbot RAG

Backend lưu tin nhắn người dùng trước khi gọi AI. Python Server truy xuất chunk, lọc theo mode, xếp hạng và tạo context. LLM sinh câu trả lời tiếng Việt; bộ làm sạch loại bỏ định dạng không phù hợp. Backend lưu tin nhắn assistant và trả cả hai tin nhắn để client cập nhật. Nếu AI lỗi, backend trả lỗi upstream để giao diện thông báo.



Hình 2.9: Sơ đồ hoạt động chatbot RAG

2.5. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng sáu collection nghiệp vụ. Các trường thời gian lưu dạng timestamp JavaScript. Trường `_destroy` được dùng ở một số collection để hỗ trợ trạng thái xóa mềm, trong khi lịch sử nhận diện và tin nhắn được xóa vật lý theo luồng hiện tại.

2.5.1. Collection users

Bảng 2.3. Thiết kế collection users

Trường	Kiểu	Ý nghĩa
<code>_id</code>	ObjectId	Định danh người dùng.
<code>email</code>	String	Email đăng nhập, phải hợp lệ.
<code>password</code>	String	Mật khẩu đã băm.
<code>username</code>	String	Tên kỹ thuật sinh từ email.
<code>displayName</code>	String	Tên hiển thị.
<code>avatar</code>	String/null	URL ảnh đại diện Cloudinary.
<code>role</code>	doctor/patient	Vai trò tài khoản.
<code>isActive</code>	Boolean	Trạng thái hoạt động; tài khoản mới mặc định true.
<code>verifyToken</code>	String/null	Mã OTP 6 số dùng để đặt lại mật khẩu.
<code>verifyTokenExpiresAt</code>	Number/null	Thời điểm OTP hết hạn.
<code>verifyTokenSentAt</code>	Number/null	Thời điểm gửi OTP gần nhất.
<code>verifyTokenAttempts</code>	Number	Số lần nhập sai OTP, mặc định 0.
<code>createdAt, updatedAt</code>	Number	Thời điểm tạo/cập nhật.

2.5.2. Collection conversations và messages

Bảng 2.4. Thiết kế collection conversations

Trường	Kiểu	Ý nghĩa
_id	ObjectId	Định danh hội thoại.
doctorId	String	ID bác sĩ.
patientId	String	ID bệnh nhân.
createdAt, updatedAt	Number	Thời điểm tạo/cập nhật.
_destroy	Boolean	Trạng thái xóa.

Bảng 2.5. Thiết kế collection messages

Trường	Kiểu	Ý nghĩa
_id	ObjectId	Định danh tin nhắn.
conversationId	String	Hội thoại sở hữu tin nhắn.
sender	doctor/patient	Vai trò người gửi.
content	String/null	Nội dung văn bản.
image	String/null	URL ảnh đính kèm.
createdAt	Number	Thời điểm gửi.
_destroy	Boolean	Trạng thái xóa.

2.5.3. Collection ai_conversations và ai_messages

Bảng 2.6. Thiết kế collection ai_conversations

Trường	Kiểu	Ý nghĩa
_id	ObjectId	Định danh hội thoại AI.
userId	String	Người sở hữu.
title	String	Tiêu đề tối đa 120 ký tự.
createdAt, updatedAt	Number	Thời điểm tạo/cập nhật.
_destroy	Boolean	Trạng thái xóa.

Bảng 2.7. Thiết kế collection ai_messages

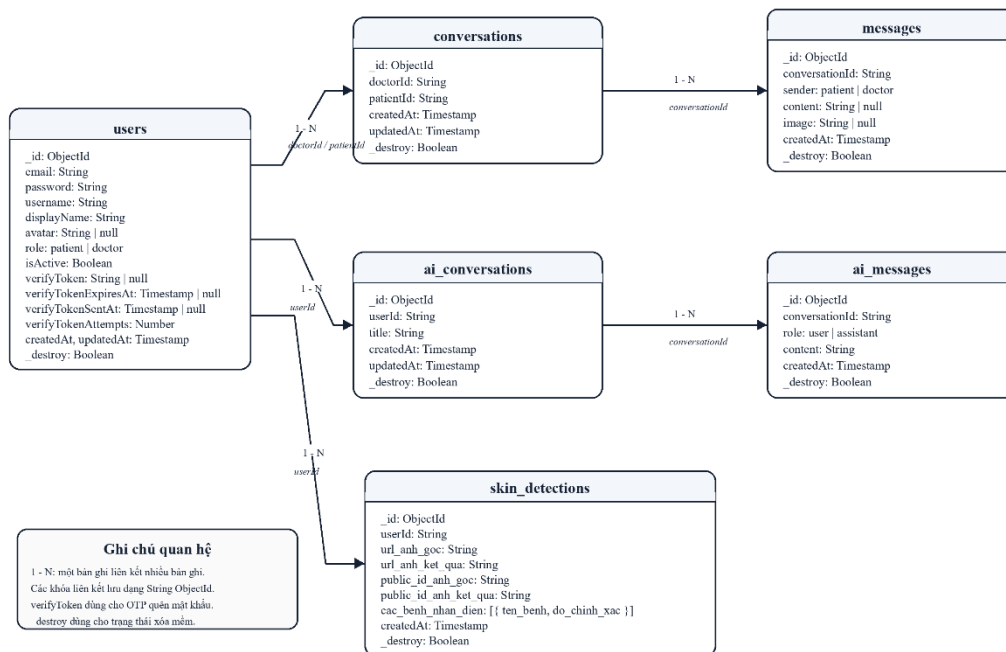
Trường	Kiểu	Ý nghĩa
conversationId	String	Hội thoại AI chứa tin nhắn.
role	user/assistant	Vai trò của tin nhắn.
content	String	Câu hỏi hoặc câu trả lời.
createdAt	Number	Thời điểm tạo.
_destroy	Boolean	Trạng thái xóa.

2.5.4. Collection skin_detections

Bảng 2.8. Thiết kế collection skin_detections

Trường	Kiểu	Ý nghĩa
userId	String	Người sở hữu lượt nhận diện.
url_anh_goc	String	URL ảnh đầu vào.
url_anh_ket_qua	String	URL ảnh đã khoanh vùng.
public_id_anh_goc	String	Public ID để xóa ảnh gốc.
public_id_anh_ket_qua	String	Public ID để xóa ảnh kết quả.
cac_benh_nhan_dien	Array	Danh sách tên bệnh và độ tin cậy.
createdAt	Number	Thời điểm nhận diện.

Hình 2.7: Sơ đồ dữ liệu MongoDB

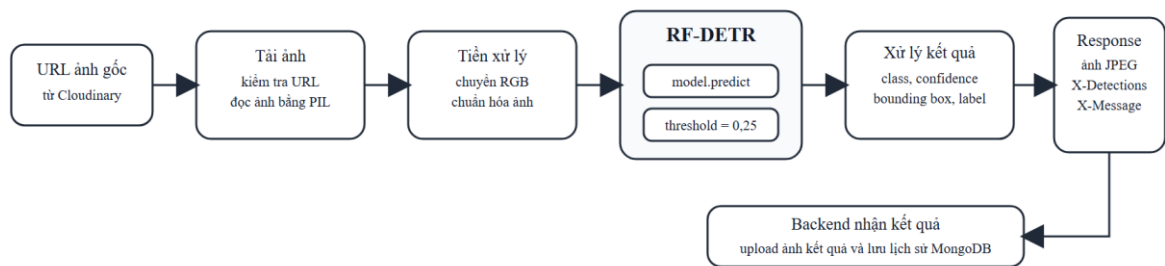


Hình 2.10: Sơ đồ dữ liệu MongoDB

2.6. Thiết kế pipeline nhận diện bệnh da liễu

Model được tải một lần khi FastAPI khởi động. Một lock bảo vệ đoạn suy luận để tránh nhiều request đồng thời truy cập model theo cách không an toàn. Ảnh tải về được chuyển RGB. Hàm predict nhận threshold 0,25; mỗi detection được chuyển thành tên lớp và confidence phần trăm. supervision tạo bounding box và label; Pillow mã hóa JPEG chất lượng 95.

API trả ảnh trong body để tránh base64 làm tăng kích thước. Metadata dự đoán được JSON hóa, URL encode và đặt trong X-Detections; thông báo nằm ở X-Detection-Message. Backend kiểm tra content-type, buffer rỗng và khả năng giải mã metadata trước khi chấp nhận kết quả.



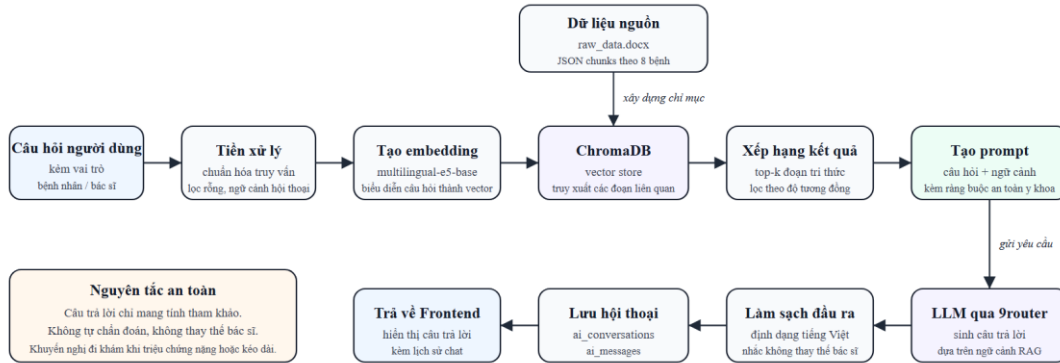
Hình 2.11: Pipeline suy luận RF-DETR

2.7. Thiết kế pipeline RAG

Nguồn raw_data.docx được chuyển thành JSON gồm 306 chunk. Mỗi chunk chứa nội dung và metadata an toàn. Khi xây dựng index, hệ thống ghép các trường có giá trị truy xuất, tạo embedding và thêm theo batch 64 phần tử vào collection dermatology_rag_chunks. Health check so sánh số chunk nguồn và số vector để phát hiện chỉ mục thiếu.

Khi có câu hỏi, hệ thống trích xuất tag bằng cách so khớp thuật ngữ đã biết, truy vấn vector với số ứng viên lớn hơn topK, sau đó tính final score bằng semantic score cộng metadata score. Các chunk không đủ priority hoặc không phù hợp mode bị loại. Câu

hỏi chứa dấu hiệu nguy hiểm hoặc điều trị được bổ sung ứng viên ưu tiên 5. Kết quả cuối cùng được khử trùng và lấy topK.



Hình 2.12: Pipeline truy xuất và sinh câu trả lời RAG

2.8. Thiết kế bảo mật và xử lý lỗi

Mật khẩu được băm bằng bcryptjs. Các route nghiệp vụ đặt authMiddleware trước controller. Backend không tin userId do client truyền mà lấy từ token. Quyền sở hữu hội thoại AI và lịch sử nhận diện được kiểm tra trong truy vấn. Hội thoại trực tiếp chỉ cho phép hai participant truy cập.

File upload được giới hạn qua middleware và validator frontend. CORS chỉ cho phép domain cấu hình; cookie production cần secure và sameSite phù hợp. Các secret MongoDB, JWT, Cloudinary, Brevo, Python Server và 9router không được ghi trực tiếp trong mã nguồn. Error middleware chuẩn hóa lỗi, còn Python Server chuyển lỗi tải ảnh, model và RAG thành HTTPException.

2.9. Kết chương

Chương 2 đã xác định yêu cầu, tác nhân, kiến trúc và thiết kế dữ liệu của hệ thống. Các luồng nhận diện và RAG được tách rõ nhưng tích hợp thông qua backend, đảm bảo người dùng chỉ truy cập dữ liệu thuộc quyền sở hữu và các chức năng AI luôn nằm trong bối cảnh cảnh báo y khoa.

CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ

3.1. Dữ liệu hình ảnh

Tập dữ liệu hình ảnh của đồ án gồm 899 ảnh thuộc tám nhóm bệnh: viêm da cơ địa, mề đay, vẩy nến, bệnh lý niêm mạc miệng, zona thần kinh, mụn cóc, giang mai và mụn trứng cá. Ảnh được tổng hợp từ Dermatology Atlas, DermNet và các kết quả tìm kiếm bổ sung trên Google Images, sau đó được sàng lọc, phân loại và gán nhãn trên Roboflow để phục vụ huấn luyện mô hình phát hiện đối tượng. Do được thu thập từ nhiều nguồn, dữ liệu có sự khác biệt về độ phân giải, ánh sáng, màu da, góc chụp, khoảng cách máy ảnh, nền ảnh và mức độ biểu hiện của tổn thương.

Bảng 3.1. Thống kê tập dữ liệu hình ảnh

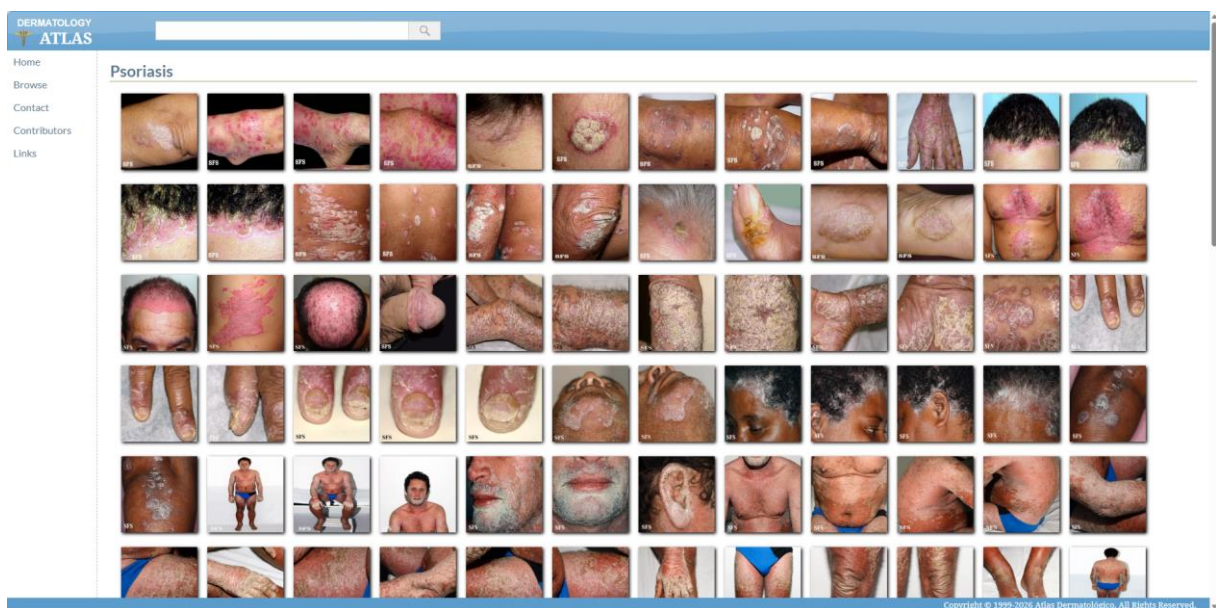
STT	Nhóm bệnh	Số ảnh
1	Viêm da cơ địa	73
2	Mề đay	90
3	Vẩy nến	207
4	Bệnh lý niêm mạc miệng	91
5	Zona thần kinh	123
6	Mụn cóc	126
7	Giang mai	90
8	Mụn trứng cá	89
	Tổng cộng	899

Bảng 3.1 cho thấy dữ liệu chưa cân bằng hoàn toàn giữa các lớp. Vẩy nến có 207 ảnh, chiếm số lượng lớn nhất, trong khi viêm da cơ địa có 73 ảnh, là lớp ít dữ liệu nhất. Sự chênh lệch này có thể làm mô hình ưu tiên các lớp phổ biến và bỏ sót lớp ít mẫu; do đó, quá trình đánh giá cần xem xét precision, recall và AP theo từng lớp bên cạnh các chỉ số tổng hợp. Khi chia dữ liệu, các ảnh trùng lặp, ảnh được cắt từ cùng một ảnh gốc

hoặc các biến thể chỉnh sửa của cùng ca bệnh phải được giữ trong cùng một tập để hạn chế rò rỉ dữ liệu.

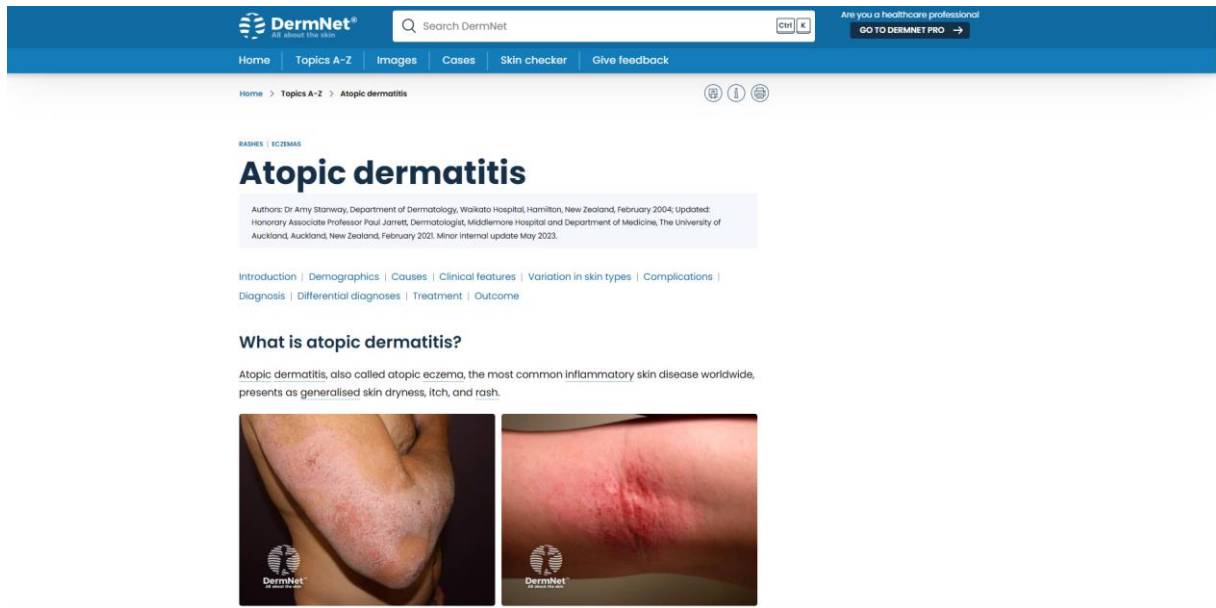
Quá trình thu thập bắt đầu bằng việc xây dựng bộ từ khóa cho từng nhóm bệnh. Ngoài tên bệnh bằng tiếng Việt, đề tài sử dụng tên tiếng Anh và các từ đồng nghĩa thường gặp để mở rộng phạm vi tìm kiếm. Kết quả được duyệt thủ công thay vì tải hàng loạt, nhằm bảo đảm ảnh thực sự thể hiện tổn thương da phù hợp với nhãn dự kiến và có đủ thông tin thị giác để khoanh vùng.

Dermatology Atlas được sử dụng như một nguồn ảnh lâm sàng ban đầu. Các ảnh được tìm theo từng chẩn đoán, kiểm tra phân mô tả đi kèm và chỉ giữ lại trường hợp có biểu hiện quan sát được tương đối rõ. Những ảnh không thuộc phạm vi tám nhóm bệnh, ảnh mô bệnh học, ảnh minh họa hoặc ảnh mà tổn thương bị che khuất quá nhiều được loại khỏi tập dữ liệu.



Hình 3.1: Nguồn ảnh từ Dermatology Atlas

DermNet là nguồn tham khảo tiếp theo nhờ cách tổ chức nội dung theo bệnh và hình thái tổn thương. Ảnh được đối chiếu với tiêu đề, mô tả và nhóm bệnh trước khi đưa vào thư mục tương ứng. Việc đối chiếu này giúp hạn chế trường hợp hình ảnh có biểu hiện gần giống nhau nhưng thuộc chẩn đoán khác, đặc biệt đối với các tổn thương dạng dát, sẩn, mụn nước hoặc vùng viêm đỏ.



Hình 3.2: Nguồn ảnh từ DermNet

Google Images được dùng để bổ sung số lượng và tăng tính đa dạng cho các lớp còn ít mẫu. Truy vấn được kết hợp giữa tên bệnh, vị trí cơ thể và các từ khóa như clinical, skin lesion hoặc dermatology. Mỗi ảnh tìm được đều được mở tại trang nguồn để kiểm tra ngữ cảnh; ảnh không xác định được nội dung, ảnh quảng cáo, ảnh ghép, ảnh có chữ che phủ vùng bệnh hoặc ảnh không đủ độ phân giải không được sử dụng.



Hình 3.3: Tìm kiếm ảnh bổ sung trên Google Images

Sau khi thu thập, dữ liệu được sàng lọc theo ba nhóm tiêu chí. Về nội dung, ảnh phải phù hợp với một trong tám lớp và vùng tổn thương phải có thể nhận biết bằng mắt. Về chất lượng, ảnh quá mờ, quá tối, vỡ hạt, bị cắt mất phần lớn tổn thương hoặc chứa quá nhiều vật cản được loại bỏ. Về tính duy nhất, các ảnh trùng hoàn toàn, ảnh đối kích

thước, lật, nén lại hoặc xuất hiện ở nhiều website được đổi chiều và chỉ giữ một bản đại diện để tránh làm sai lệch kết quả đánh giá.

Các ảnh đạt yêu cầu được đổi tên theo quy ước thống nhất và đưa vào thư mục lớp tương ứng. Định dạng ảnh được kiểm tra để bảo đảm có thể đọc bằng pipeline huấn luyện; ảnh lỗi hoặc sai kênh màu được chuyển về định dạng thông dụng. Nhãn lớp được chuẩn hóa thành một danh sách cố định, tránh tình trạng cùng một bệnh nhưng tồn tại nhiều cách viết, dấu tiếng Việt hoặc ký hiệu khác nhau.

Dữ liệu sau sàng lọc được tải lên Roboflow để gán nhãn phát hiện đối tượng. Với mỗi ảnh, bounding box được vẽ bao quanh vùng tổn thương nhìn thấy rõ và gán đúng tên lớp. Nếu ảnh có nhiều vùng tổn thương tách biệt, có thể tạo nhiều bounding box; tuy nhiên, khung không được mở rộng quá mức sang vùng da lành và cũng không được cắt mất phần biểu hiện quan trọng. Các trường hợp tổn thương lan tỏa được khoanh theo vùng đại diện có ranh giới quan sát được, bảo đảm cách gán nhãn nhất quán giữa các ảnh.



Hình 3.4: Gán nhãn vùng tổn thương trên Roboflow

Sau vòng gán nhãn đầu tiên, toàn bộ ảnh được rà soát lại theo từng lớp trên Roboflow. Các khung sai lớp, bỏ sót tổn thương, chồng lấn không cần thiết hoặc có kích thước bất thường được chỉnh sửa. Bước kiểm tra chéo này đặc biệt quan trọng với ảnh có nhiều tổn thương nhỏ và các lớp có hình thái gần nhau. Chỉ những ảnh có nhãn hợp lệ mới được đưa vào phiên bản dữ liệu dùng cho huấn luyện.

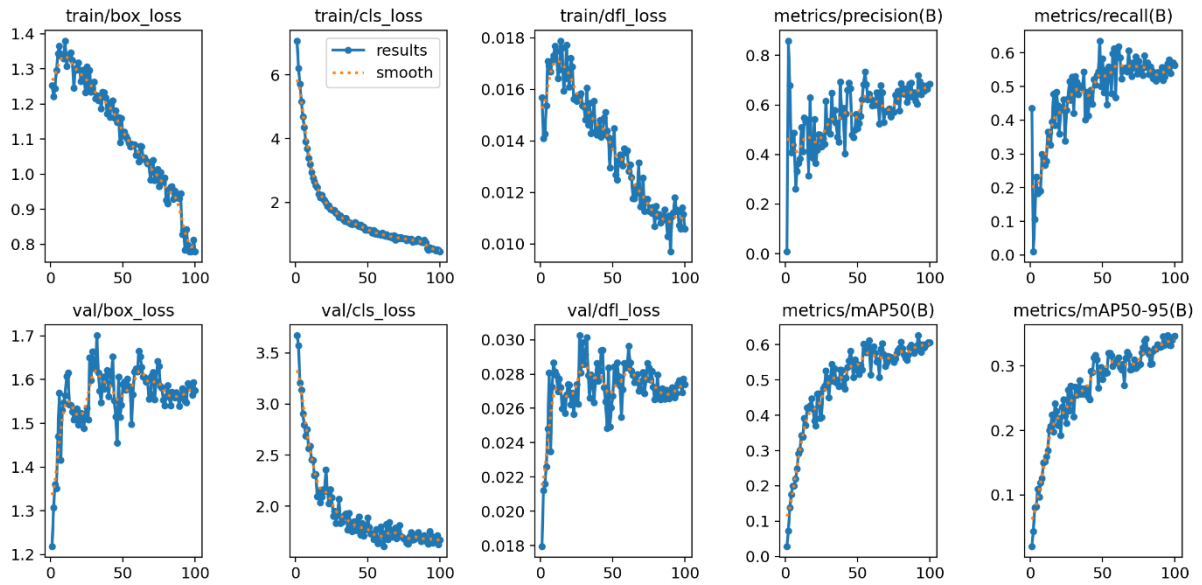
Cuối cùng, dữ liệu được chia thành các tập huấn luyện, validation và kiểm thử với tỉ lệ 80% 10% và 10%. Việc chia được thực hiện sau bước loại trùng và có xét đến phân bố lớp để mỗi tập đều chứa đại diện của tám nhóm bệnh. Tập huấn luyện được dùng để tối ưu tham số mô hình, tập validation phục vụ theo dõi quá trình học và lựa chọn checkpoint, còn tập kiểm thử được giữ độc lập để đánh giá khả năng khái quát. Quy trình trên tạo ra một tập dữ liệu có nguồn gốc rõ ràng, nhãn nhất quán và phù hợp với bài toán phát hiện vùng tổn thương da trong phạm vi đề tài.

3.2. Đánh giá mô hình YOLO và RF-DETR

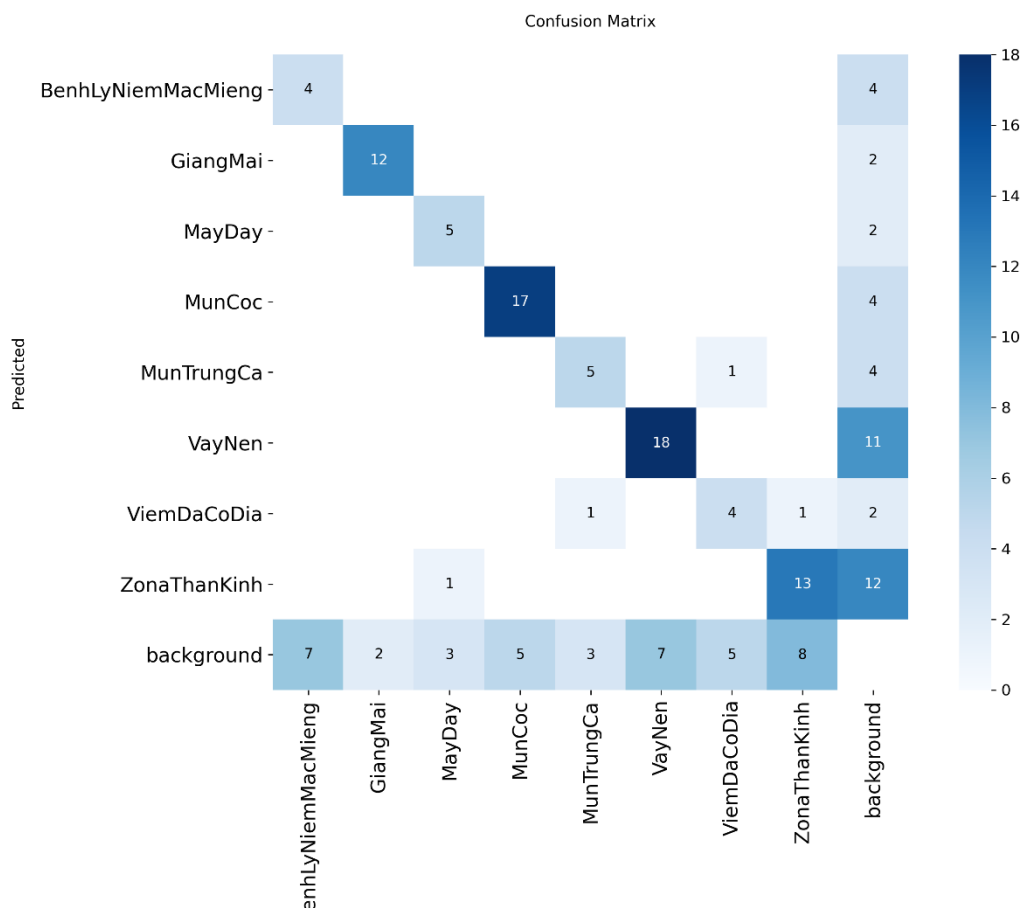
Hai mô hình được đặt trong cùng bài toán để đánh giá khả năng phát hiện tổn thương. Các chỉ số cần tổng hợp gồm precision, recall, F1-score, mAP@0.5, mAP@0.5:0.95, thời gian suy luận và kích thước checkpoint. Ngoài số liệu tổng hợp, cần quan sát confusion matrix, kết quả theo lớp và ảnh ngoài tập dữ liệu.

Bảng 3.2. So sánh kết quả YOLO và RF-DETR

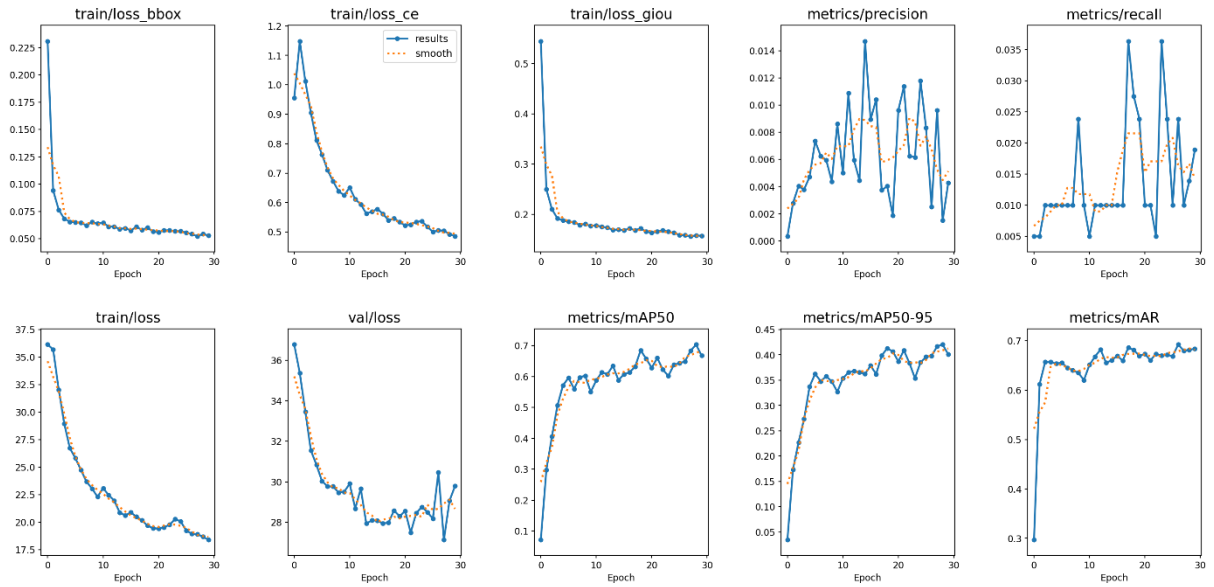
Chỉ số	YOLO	RF-DETR	Nhận xét
Precision	82,15%	85,88%	YOLO có precision nhìn hơn, ít dự đoán nhầm hơn ở ngưỡng đánh giá tối ưu.
Recall	63,56%	76,84%	RF-DETR phát hiện được nhiều đối tượng hơn, giảm tỷ lệ bỏ sót.
F1-score	62,96%	79.23%	RF-DETR đạt F1 cao hơn nhờ cân bằng tốt giữa precision và recall.
mAP@0.5	62,63%	70,35%	RF-DETR đạt mAP@0.5 cao hơn.
mAP@0.5:0.95	34,72%	42,04%	RF-DETR tốt hơn ở dải IoU 0.5:0.95.
Kích thước checkpoint	Khoảng 5,15 MB	Khoảng 127,2 MB	RF-DETR lớn hơn đáng kể.
Khả năng tích hợp	Chưa tích hợp chính thức	Đã tích hợp vào FastAPI	RF-DETR là mô hình suy luận chính của hệ thống hiện tại.



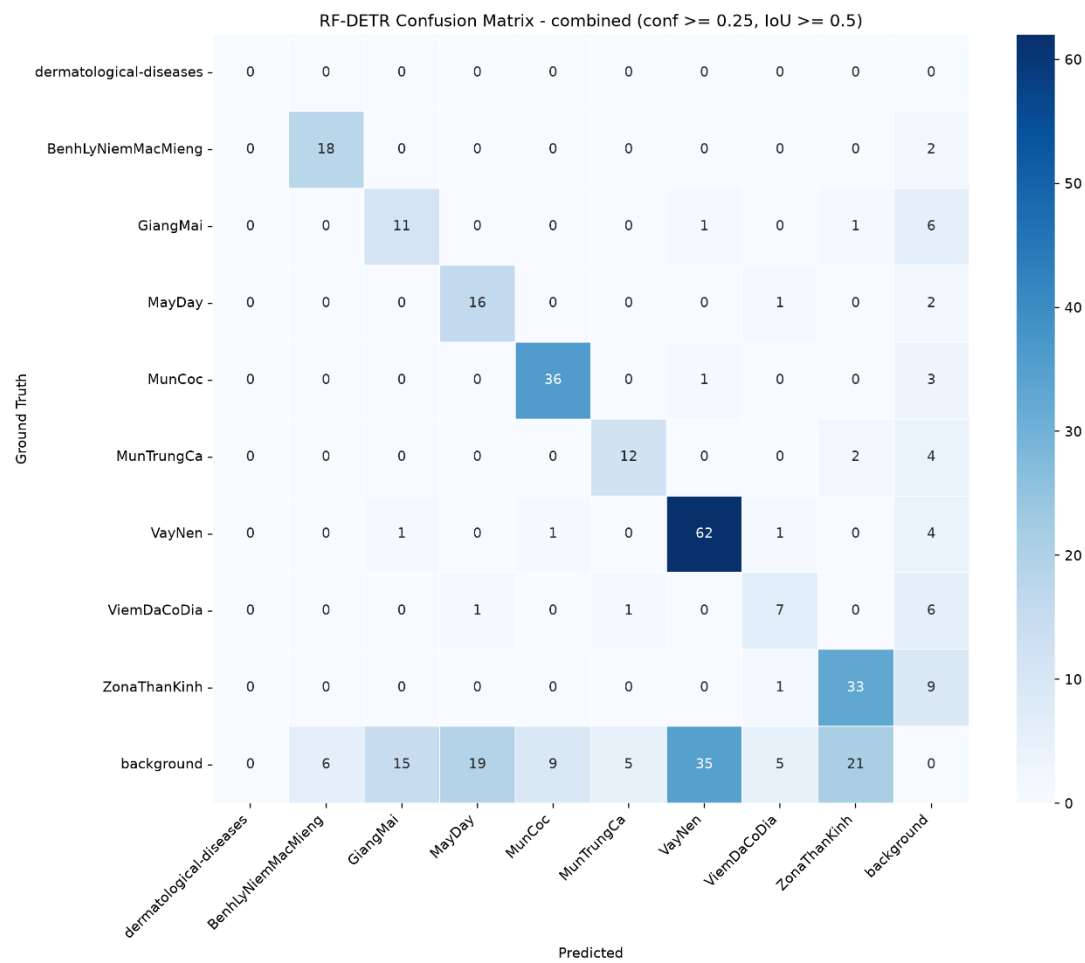
Hình 3.5: Biểu đồ huấn luyện YOLO



Hình 3.6: Ma trận nhầm lẫn YOLO



Hình 3.7: Biểu đồ huấn luyện RF-DETR



Hình 3.8: Ma trận nhầm lẫn RF-DETR

Nhận xét:

Để đánh giá hiệu quả của mô hình nhận diện tổn thương da, đề tài tiến hành huấn luyện và so sánh hai kiến trúc phổ biến là RF-DETR và YOLO26 trên cùng một tập dữ liệu. Kết quả thực nghiệm cho thấy RF-DETR đạt hiệu năng vượt trội so với YOLO26 ở hầu hết các chỉ số đánh giá quan trọng. Cụ thể, RF-DETR đạt giá trị mAP@50 là 70,35% và mAP@50-95 là 42,04%, trong khi YOLO26 chỉ đạt lần lượt 60,09% và 34,72%. Điều này cho thấy RF-DETR không chỉ có khả năng phát hiện đối tượng chính xác hơn mà còn định vị vùng tổn thương tốt hơn trên nhiều ngưỡng IoU khác nhau.

Trong quá trình huấn luyện, RF-DETR đạt kết quả tốt nhất sau khoảng 28 epoch và tiếp tục cải thiện ổn định trong suốt quá trình học. Ngược lại, YOLO26 cần tới gần 100 epoch để hội tụ nhưng vẫn cho kết quả thấp hơn. Sự chênh lệch hơn 10% ở chỉ số mAP@50 và hơn 7% ở chỉ số mAP@50-95 cho thấy RF-DETR có khả năng học đặc trưng của dữ liệu da liễu hiệu quả hơn.

Mặc dù YOLO26 có ưu điểm về tốc độ suy luận và phù hợp với các ứng dụng yêu cầu xử lý thời gian thực, RF-DETR lại thể hiện ưu thế rõ rệt về độ chính xác. Đối với bài toán hỗ trợ nhận diện bệnh da liễu trong đề tài, yếu tố độ chính xác được ưu tiên hơn tốc độ xử lý, bởi kết quả dự đoán có ảnh hưởng trực tiếp đến chất lượng thông tin tham khảo cung cấp cho người dùng. Vì vậy, RF-DETR được lựa chọn làm mô hình chính của hệ thống nhờ khả năng phát hiện và định vị tổn thương da tốt hơn so với YOLO26.

Phiên bản hệ thống hiện tại sử dụng RF-DETR làm mô hình suy luận chính. Quyết định cuối cùng phải được giải thích bằng số liệu ở Bảng 3.2. Bên cạnh mAP, cần xem xét recall đối với lớp có ít dữ liệu, mức độ khoanh vùng đúng và số dự đoán sai trên ảnh thực tế. Nếu RF-DETR có chất lượng phát hiện tốt hơn nhưng chậm hơn, sự đánh đổi vẫn có thể chấp nhận vì ứng dụng xử lý từng ảnh và không yêu cầu video thời gian thực.

Kích thước checkpoint RF-DETR khoảng 127,2 MB, lớn hơn nhiều so với checkpoint YOLO khoảng 5,15 MB. Điều này làm tăng thời gian khởi động và yêu cầu bộ nhớ.

Do đó, khi triển khai cần tải model một lần, dùng health check và cân nhắc giới hạn đồng thời. Nếu hạ tầng yếu hoặc cần phản hồi rất nhanh, YOLO vẫn là phương án dự phòng đáng xem xét.

3.3. Dữ liệu tri thức và quy trình xây dựng chatbot RAG

Hệ thống chatbot được xây dựng theo kiến trúc Retrieval-Augmented Generation (RAG). Quy trình triển khai gồm hai pha: pha chuẩn bị và lập chỉ mục dữ liệu được thực hiện khi xây dựng hoặc cập nhật kho tri thức; pha truy xuất và sinh câu trả lời được thực hiện cho mỗi câu hỏi của người dùng.

Bước 1. Chuẩn bị nguồn văn bản y khoa

Nguồn đầu vào là tệp raw_data.docx gồm nội dung của tám nhóm bệnh. Tài liệu được tổ chức theo ba cấp: tiêu đề bệnh, mục chuyên môn như Tổng quan, Triệu chứng, Chẩn đoán hoặc Điều trị, và chủ đề con bên trong từng mục. Trước khi xử lý, nội dung phải được kiểm tra nguồn, loại bỏ phần ngoài phạm vi đề tài và không đưa thông tin chưa được đối chiếu vào kho tri thức.

Bước 2. Làm sạch và chuẩn hóa dữ liệu

Văn bản được đọc theo thứ tự các đoạn trong tài liệu. Hệ thống chuẩn hóa Unicode tiếng Việt, khoảng trắng, dấu câu và tên bệnh. Các tiêu đề viết hoa được nhận diện để xác định section, còn các tiêu đề nhỏ được chuyển thành subtopic. Bước này giúp cùng một khái niệm không tồn tại dưới nhiều cách viết và giữ được quan hệ giữa nội dung với bệnh tương ứng.

Bước 3. Chia văn bản thành chunk

Chunk là một đoạn tri thức tương đối độc lập về ngữ nghĩa và là đơn vị nhỏ nhất được lưu, truy xuất và đưa vào mô hình ngôn ngữ. Việc chia chunk không thực hiện đơn thuần theo số ký tự mà ưu tiên ranh giới câu, bệnh, section và subtopic. Các câu cùng chủ đề được ghép thành đoạn khoảng 85–700 ký tự như dữ liệu hiện tại. Chunk quá ngắn dễ mất ngữ cảnh; chunk quá dài có thể chứa nhiều ý khác nhau, làm giảm độ chính xác khi truy xuất.

Bước 4. Xây dựng JSON và metadata

Kết quả được lưu trong dermatology_rag_chunks.json. Ngoài content là nội dung chính, mỗi chunk có metadata để hỗ trợ tìm kiếm, phân quyền và an toàn. Ví dụ rút gọn của một chunk như sau:

```
{ "id": "viem_da_co_dia.tong_quan.tong_quan_ket_luan.001", "disease":  
"Viêm da cơ địa", "section": "Tổng quan", "subtopic":  
"tổng_quan_kết_luận", "content_type":  
"background_summary", "access_level": "patient_safe", "risk_level":  
"medium", "requires_doctor": false, "retrieval_priority": 1, "retrieval_role":  
"background_low_recall", "symptom_tags": ["ngứa", "khô  
da"], "body_site_tags": ["mặt", "miệng"], "content": "...", "safe_answer":  
"..." }
```

Access level quy định phạm vi sử dụng. patient_safe là nội dung có thể cung cấp trực tiếp cho bệnh nhân dưới dạng giáo dục sức khỏe; mixed là nội dung có phần công khai nhưng phải trình bày thận trọng; clinician_only là nội dung chuyên môn, thuốc, chẩn đoán hoặc xử trí cần bác sĩ đánh giá. Trường requires_doctor đánh dấu nội dung cần khuyến nghị thăm khám, còn risk_level mô tả mức rủi ro medium hoặc high.

Các trường an toàn bổ sung gồm safe_answer, chứa cách diễn đạt an toàn có thể dùng khi tạo context, và do_not_answer, chứa các giới hạn như không chẩn đoán xác định, không kê đơn và không hướng dẫn liều thuốc kê đơn. Audience mô tả đối tượng chính và mức hiển thị public, guarded hoặc restricted.

Content type mô tả bản chất tri thức: background_summary là tổng quan; medical_reference là kiến thức chuyên môn; symptom_profile mô tả biểu hiện; management trình bày chăm sóc hoặc điều trị; differential_diagnosis phục vụ chẩn đoán phân biệt. Với chunk chẩn đoán phân biệt, possible_confusions và distinguishing_features lưu các bệnh dễ nhầm cùng đặc điểm phân biệt.

Retrieval priority có năm mức. Mức 1, background_low_recall, là kiến thức nền ít cần xuất hiện; mức 2, clinician_support, hỗ trợ chuyên môn; mức 3, supporting_context,

bổ sung ngữ cảnh; mức 4, `primary_answer`, trực tiếp trả lời ý định phổ biến; mức 5, `safety_critical`, chứa dấu hiệu nguy hiểm, thuốc nguy cơ cao hoặc trường hợp cần chuyển khám. Mức ưu tiên không thay thế độ tương đồng ngữ nghĩa mà được dùng khi lọc và tái xếp hạng.

Các nhóm tag gồm `symptom_tags` cho triệu chứng, `body_site_tags` cho vị trí, `trigger_tags` cho yếu tố khởi phát, `treatment_tags` cho điều trị, `red_flag_tags` cho dấu hiệu cảnh báo và `population_tags` cho nhóm đối tượng như trẻ em hoặc phụ nữ mang thai. Aliases lưu tên đồng nghĩa; questions lưu câu hỏi gợi ý; `boost_terms` bổ sung từ khóa quan trọng; `structured_content` tách tóm tắt, sự kiện chính và điều kiện cần chuyển khám.

Bước 5. Vector hóa và lưu vào ChromaDB

Embedding là phép biến đổi văn bản thành một vector số biểu diễn ý nghĩa ngữ nghĩa. Đề tài sử dụng `intfloat/multilingual-e5-base` vì mô hình hỗ trợ tiếng Việt và truy xuất đa ngôn ngữ. Văn bản dùng để tạo embedding không chỉ gồm content mà ghép title, disease, aliases, section, subtopic, questions, `boost_terms`, `symptom_tags`, `body_site_tags` và content. Cách ghép này giúp câu hỏi dùng từ đồng nghĩa vẫn có khả năng tìm thấy chunk phù hợp.

Các vector được lưu trong collection `dermatology_rag_chunks` của ChromaDB cùng ID, document và metadata đã làm phẳng. Collection sử dụng cosine distance. Cosine similarity đo mức gần nhau về hướng giữa vector câu hỏi và vector chunk; hai văn bản có ý nghĩa gần nhau thường có cosine similarity cao, ngay cả khi không sử dụng hoàn toàn cùng từ khóa.

Chỉ mục được xây dựng theo batch 64 chunk. Khi dữ liệu thay đổi, chạy lại lệnh `rebuild` để xóa collection cũ và tạo chỉ mục mới:

```
cd D:\Ki10\CapstoneProject\python-server python -m pip install -r requirements.txt python rag_chatbot.py --rebuild-index
```

Các biến cấu hình không chứa khóa bí mật được đặt trong .env như sau. NINE_ROUTER_API_KEY phải được cấu hình riêng trên máy triển khai và không đưa vào báo cáo hoặc mã nguồn.

```
RAG_TOP_K=6 RAG_EMBEDDING_MODEL=intfloat/multilingual-e5-base
RAG_CHROMA_DIR=data/chroma_db
NINE_ROUTER_BASE_URL=http://localhost:20128/v1
NINE_ROUTER_MODEL=cx/gpt-5.5
```

Bước 6. Truy xuất và tái xếp hạng

Khi nhận câu hỏi, hệ thống thêm tiền tố query: trước nội dung để tạo embedding truy vấn. TopK là số chunk cuối cùng được giữ lại làm context; mặc định $K = 6$ và API cho phép từ 1 đến 12. K quá nhỏ có thể thiếu thông tin, còn K quá lớn làm context dài và đưa thêm nội dung ít liên quan.

Hệ thống không lấy ngay đúng K kết quả mà truy vấn trước tối thiểu 20 ứng viên hoặc $\text{topK} \times 5$ ứng viên. Với mỗi ứng viên, semantic score được tính từ khoảng cách cosine. Metadata score được cộng theo retrieval_priority và mức trùng khớp của bệnh, triệu chứng, vị trí, dấu hiệu nguy hiểm, điều trị, yếu tố khởi phát và nhóm đối tượng. Final score bằng semantic score cộng metadata score; kết quả sau đó được loại trùng và lấy K chunk có điểm cao nhất.

Ở chế độ patient, hệ thống loại chunk không phù hợp, chỉ giữ priority từ 3 trở lên và không cung cấp clinician_only, ngoại trừ chunk mức 5 có dấu hiệu nguy hiểm hoặc điều trị trùng với câu hỏi; nội dung này vẫn được làm sạch trước khi sử dụng. Ở chế độ doctor, hệ thống cho phép priority từ 2 trở lên và có thể truy xuất clinician_only. Khi câu hỏi chứa red flag hoặc tên thuốc, các chunk safety_critical phù hợp được bổ sung để tránh bỏ sót cảnh báo.

Bước 7. Tạo context và sinh câu trả lời

Các chunk cuối cùng được ghép thành context gồm bệnh, section, priority, requires_doctor, dấu hiệu cảnh báo, nội dung và safety note. Context cùng câu hỏi và

mode được gửi đến mô hình ngôn ngữ qua giao diện tương thích OpenAI của 9router. System prompt buộc mô hình chỉ dùng context, trả lời tiếng Việt, không chẩn đoán chắc chắn, không kê đơn hoặc đưa liều thuốc kê đơn, đồng thời khuyên thăm khám khi có dấu hiệu nguy hiểm.

Có thể kiểm tra riêng bước truy xuất và toàn bộ luồng hỏi đáp bằng các lệnh sau:

```
python rag_chatbot.py --search "mụn trứng cá viêm ở mặt" --mode patient --  
top-k 6 python rag_chatbot.py --ask "Khi nào mụn trứng cá cần đi khám?" --  
mode patient --top-k 6 python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Sau khi server khởi động, GET /rag/health kiểm tra trạng thái dữ liệu, collection, embedding model và cấu hình LLM. API hỏi đáp nhận POST /chat với message, mode và topK:

```
{ "message": "Khi nào tổn thương da cần đi khám?", "mode":  
"patient", "topK": 6 }
```

Phản hồi gồm answer, mode, safetyFlags và sources. Sources chỉ cung cấp metadata truy vết như ID, bệnh, section, access level, requiresDoctor và điểm xếp hạng; người dùng cuối chỉ nhìn thấy câu trả lời đã được làm sạch. Nếu không tìm được chunk phù hợp, hệ thống yêu cầu người dùng mô tả rõ hơn thay vì tạo câu trả lời ngoài kho tri thức.

Thống kê kho tri thức

Sau quy trình trên, nguồn tri thức được xử lý thành 306 chunk. Mỗi chunk có nội dung, metadata truy xuất và quy tắc an toàn. Dữ liệu gồm 131 chunk patient_safe, 139 chunk clinician_only và 36 chunk mixed; trong đó 107 chunk được đánh dấu requires_doctor.

Bảng 3.3. Thống kê dữ liệu tri thức RAG

STT	Nhóm bệnh	Số chunk
1	Viêm da cơ địa	45
2	Mày đay	38
3	Vảy nến	58
4	Mụn trứng cá	70
5	Giang mai	9
6	Bệnh lý niêm mạc miệng	28
7	Zona thần kinh	30
8	Mụn cóc	28
	Tổng cộng	306

Số chunk giữa các bệnh khác nhau do độ dài và cấu trúc nguồn. Giang mai chỉ có chín chunk, do đó khả năng trả lời chi tiết có thể thấp hơn các nhóm khác. Đây là hạn chế cần ưu tiên khi mở rộng kho tri thức.

3.4. Môi trường triển khai

Bảng 3.4. Môi trường và công nghệ triển khai

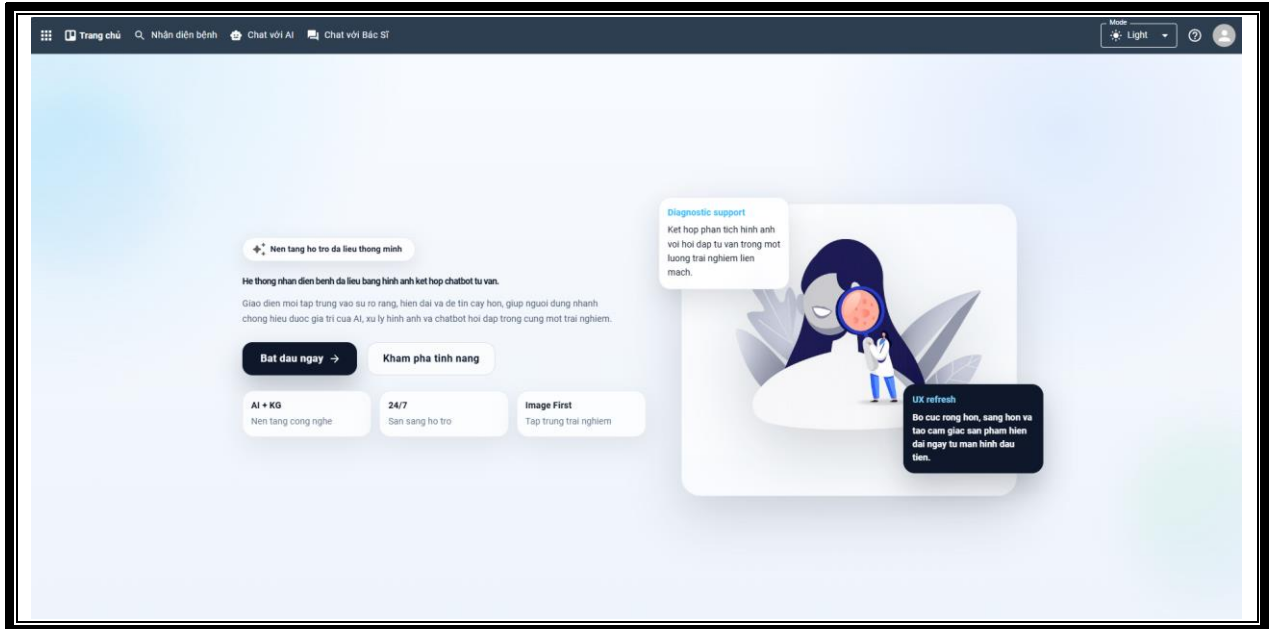
Thành phần	Công nghệ chính	Vai trò
Frontend	React 19, Vite 8, Material UI 7	Giao diện và tương tác người dùng.
Quản lý trạng thái	Redux Toolkit, Redux Persist	Người dùng, hội thoại, tin nhắn.
HTTP Client	Axios	Gọi REST API và làm mới token.
Backend	Node.js, Express 5	Nghiệp vụ, xác thực và điều phối dịch vụ.
Realtime	Socket.IO 4	Chat bác sĩ - bệnh nhân.
Cơ sở dữ liệu	MongoDB	Lưu tài khoản, hội thoại và lịch sử.
Lưu trữ ảnh	Cloudinary	Ảnh đại diện, chat và nhận diện.
Email	Brevo	Gửi mã OTP 6 số để đặt lại mật khẩu.
AI Server	Python, FastAPI, Uvicorn	Suy luận RF-DETR và RAG.
Vector DB	ChromaDB	Lưu và truy vấn embedding.
Embedding	multilingual-e5-base	Mã hóa câu hỏi và tài liệu.
LLM Gateway	9router	Sinh câu trả lời từ context.

Ba dịch vụ được chạy độc lập. Frontend dùng `npm run dev` hoặc `npm run build`. Backend dùng `npm run dev` ở môi trường phát triển, kết nối MongoDB trước khi mở cổng HTTP. Python Server chạy Uvicorn tại cổng cấu hình. Backend tham chiếu Python Server thông qua biến `PYTHON_SERVER_URL`.

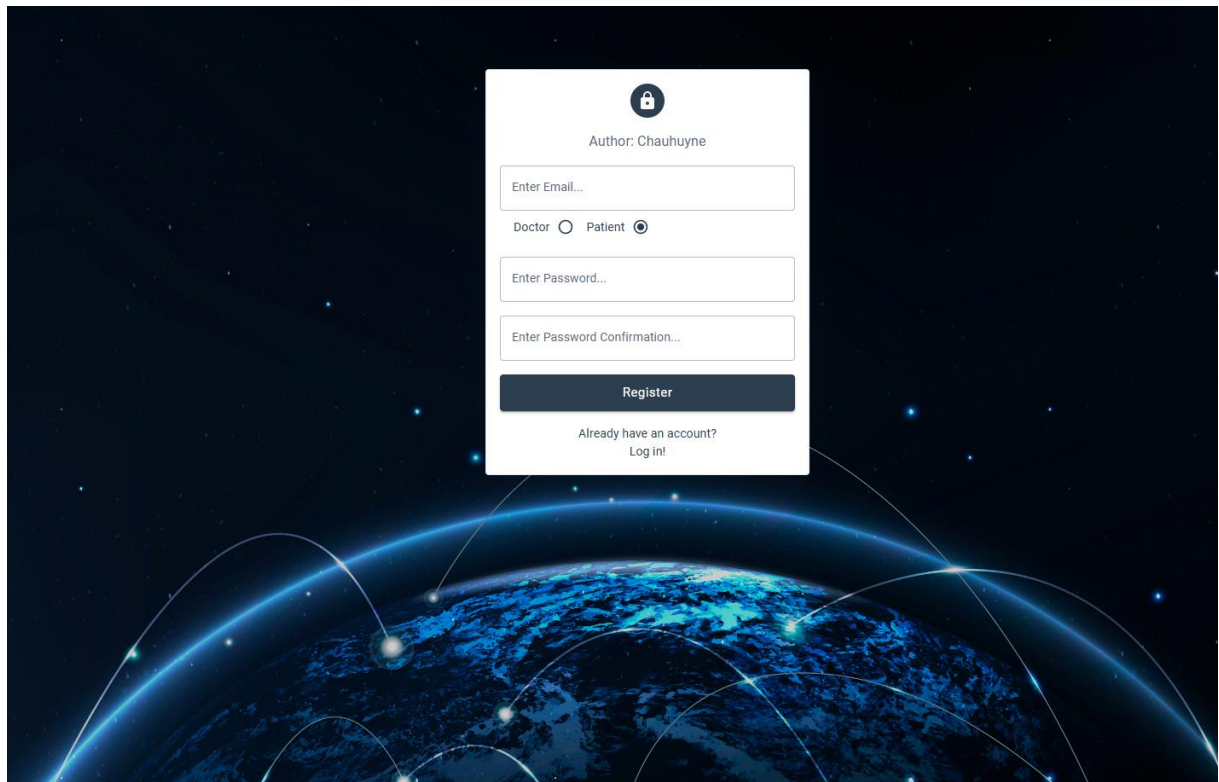
3.5. Triển khai chức năng tài khoản

Biểu mẫu đăng ký kiểm tra email, mật khẩu và vai trò. Backend dùng Joi để kiểm tra lại nhằm tránh phụ thuộc vào client. Mật khẩu hợp lệ được băm và tài khoản được tạo với `isActive=true`, `verifyToken=null` nên người dùng có thể đăng nhập ngay. Trang quên mật khẩu gồm ba bước: gửi OTP tới email, xác nhận OTP và nhập mật khẩu mới. Backend cung cấp ba endpoint `request`, `verify` và `reset`; áp dụng thời hạn 10 phút, thời gian chờ gửi lại 60 giây và giới hạn năm lần nhập sai.

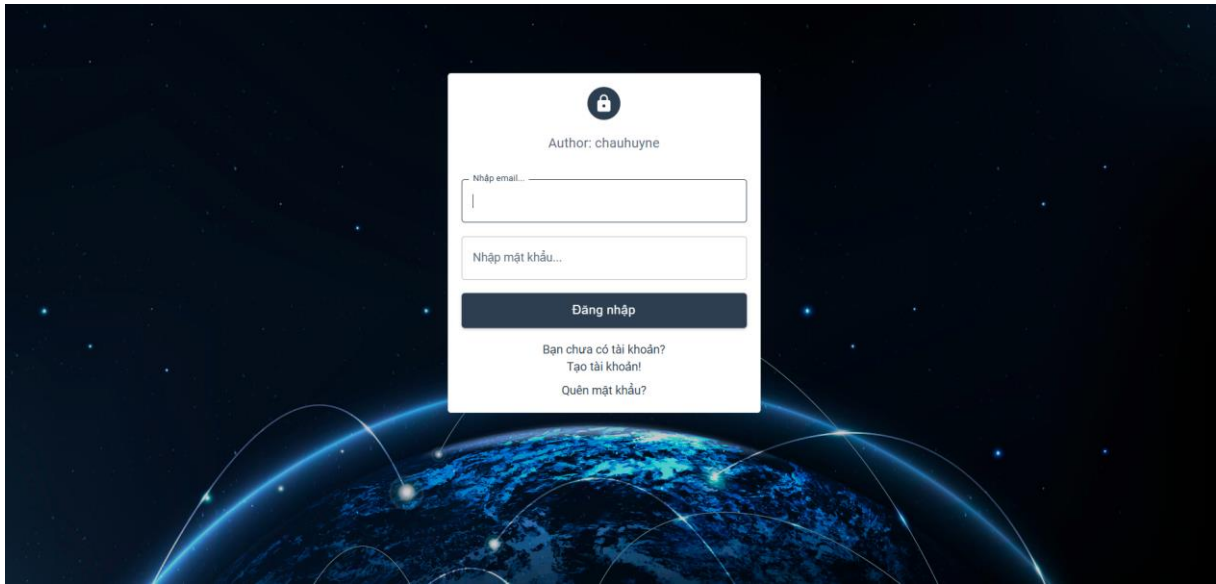
Khi đăng nhập, backend cấp access token và refresh token. Frontend lưu trạng thái người dùng bằng Redux, điều hướng qua ProtectedRoute và gọi API profile để xác minh phiên. Cài đặt tài khoản cho phép cập nhật displayName, avatar và mật khẩu; các trường email, username, role và createdAt bị loại khỏi dữ liệu cập nhật.



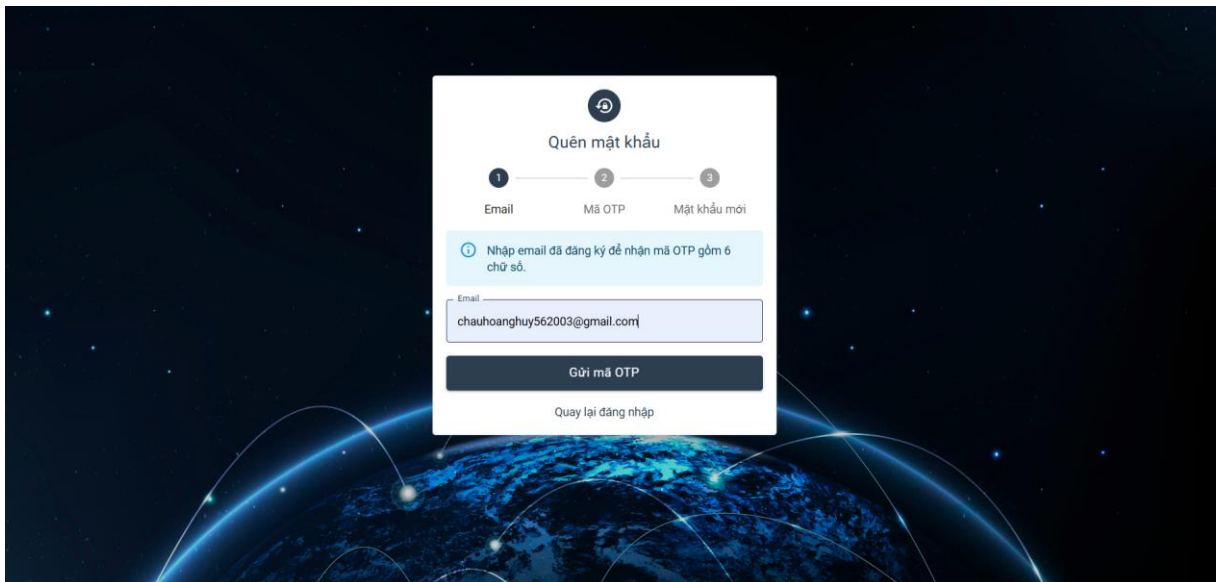
Hình 3.9: Giao diện trang chủ



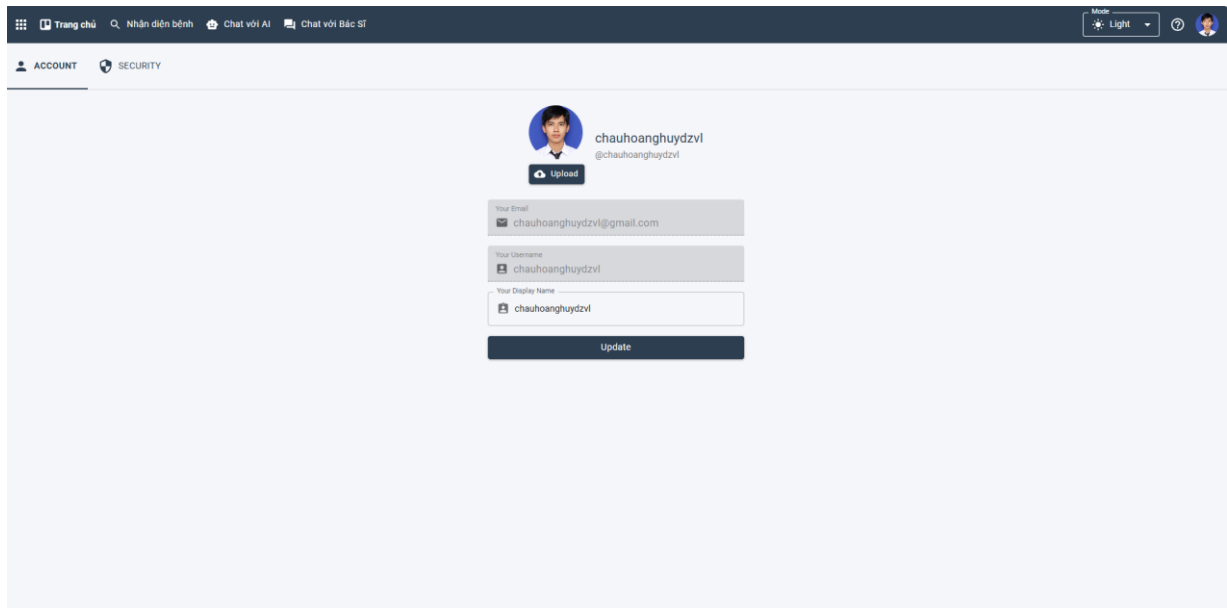
Hình 3.10: Giao diện đăng ký



Hình 3.11: Giao diện đăng nhập



Hình 3.12: Giao diện quên mật khẩu bằng OTP



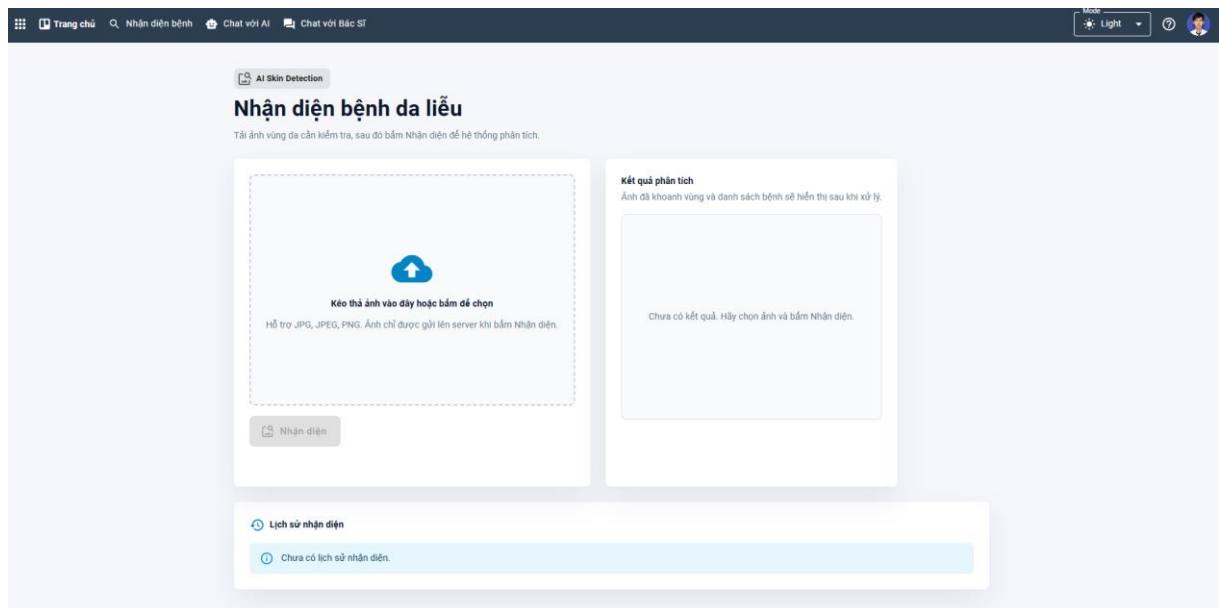
Hình 3.13: Giao diện quản lý tài khoản

3.6. Triển khai chức năng nhận diện

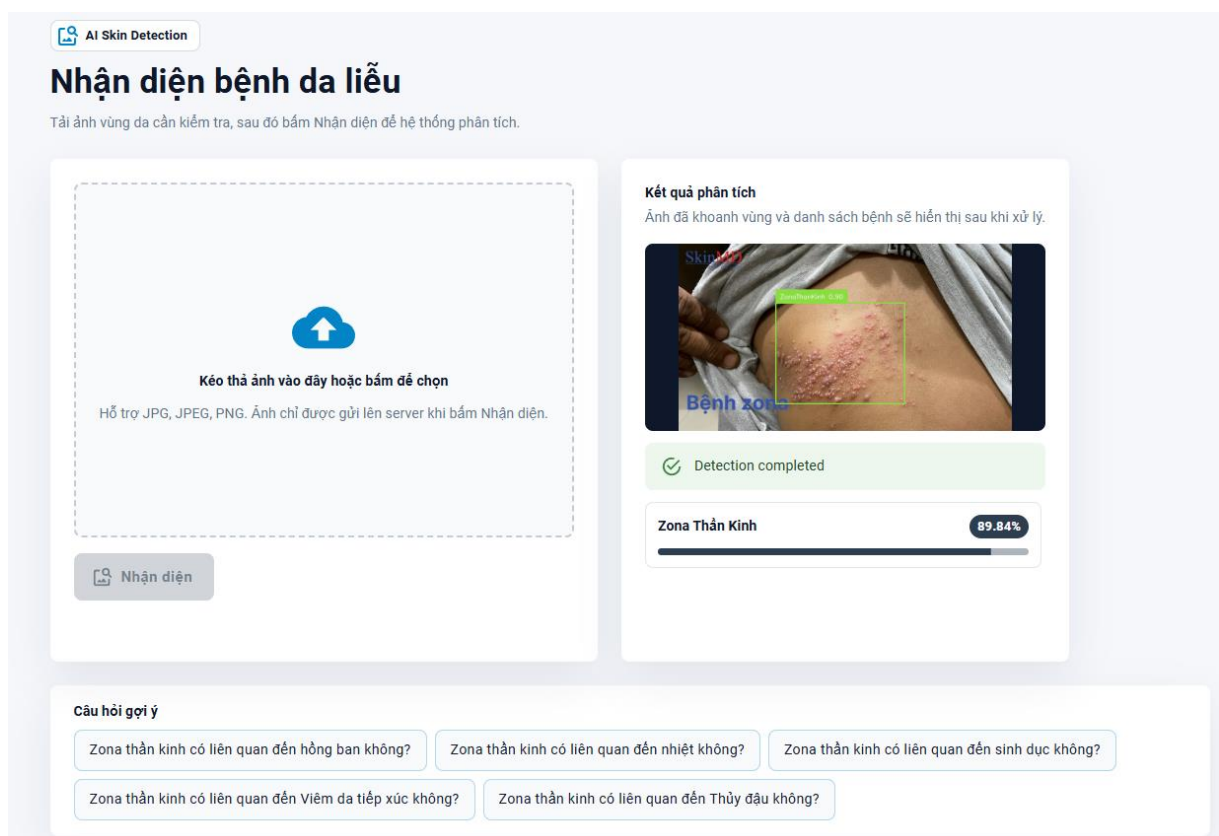
Trang nhận diện hỗ trợ chọn hoặc kéo thả ảnh. File chỉ được lưu trong trình duyệt để preview cho đến khi người dùng nhấn Nhận diện. Trong thời gian xử lý, nút bị vô hiệu hóa và hiển thị trạng thái loading. Kết quả gồm ảnh đã khoanh vùng, thông báo và danh sách detection được sắp xếp theo dữ liệu server.

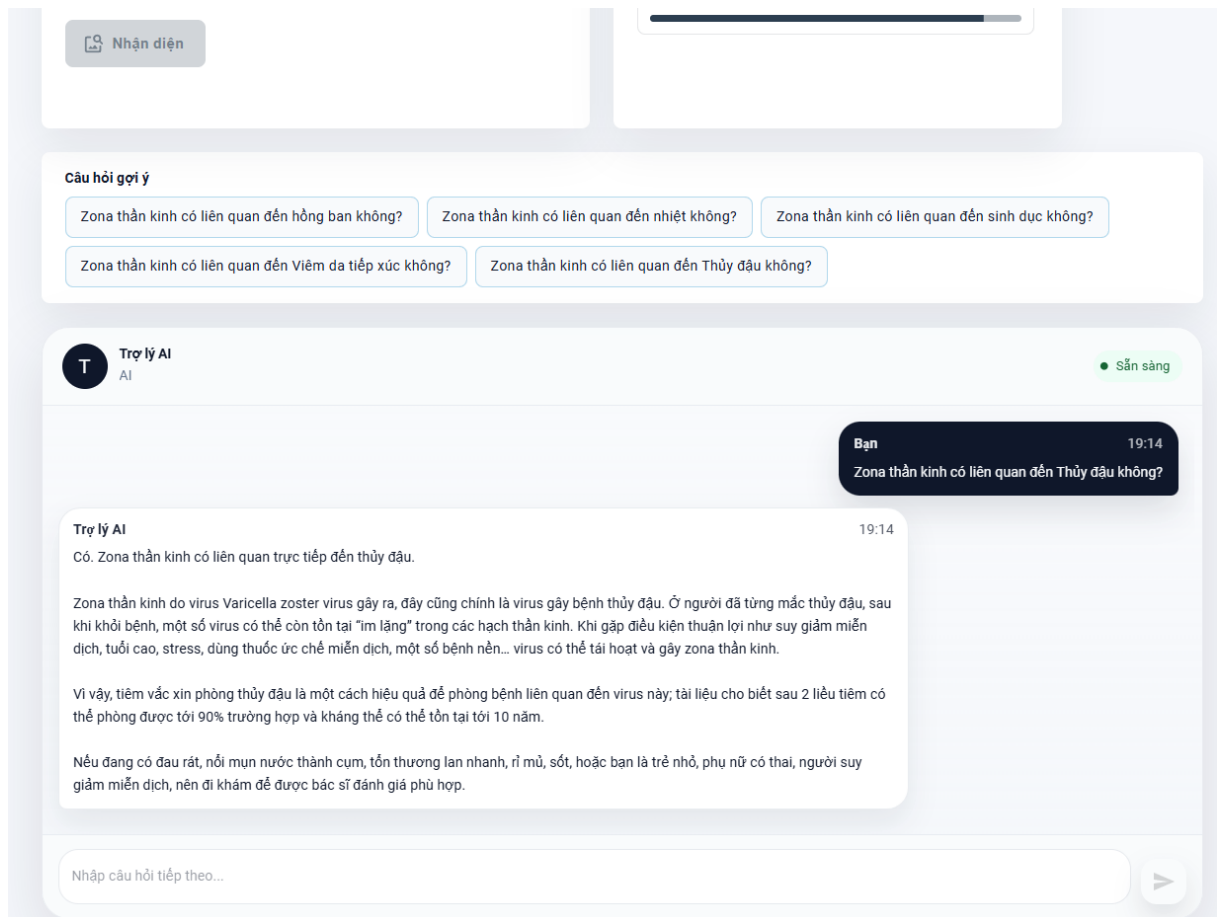
Backend upload ảnh gốc vào thư mục `CP_skin_detections/originals`. Python Server tải ảnh với timeout 20 giây, kiểm tra content và chuyển RGB. Sau suy luận, backend upload ảnh JPEG kết quả vào `CP_skin_detections/results` và lưu URL, public ID, danh sách bệnh, confidence và thời gian. Nếu không có detection, bản ghi vẫn thể hiện kết quả không phát hiện.

Lịch sử được tải khi trang khởi tạo, sắp xếp mới nhất trước. Người dùng có thể mở ảnh trong dialog, thay đổi zoom từ 0,5 đến 4 lần và xóa một mục sau xác nhận. Backend kiểm tra `detectionId` thuộc `userId`, xóa hai ảnh Clouinary rồi xóa bản ghi MongoDB.

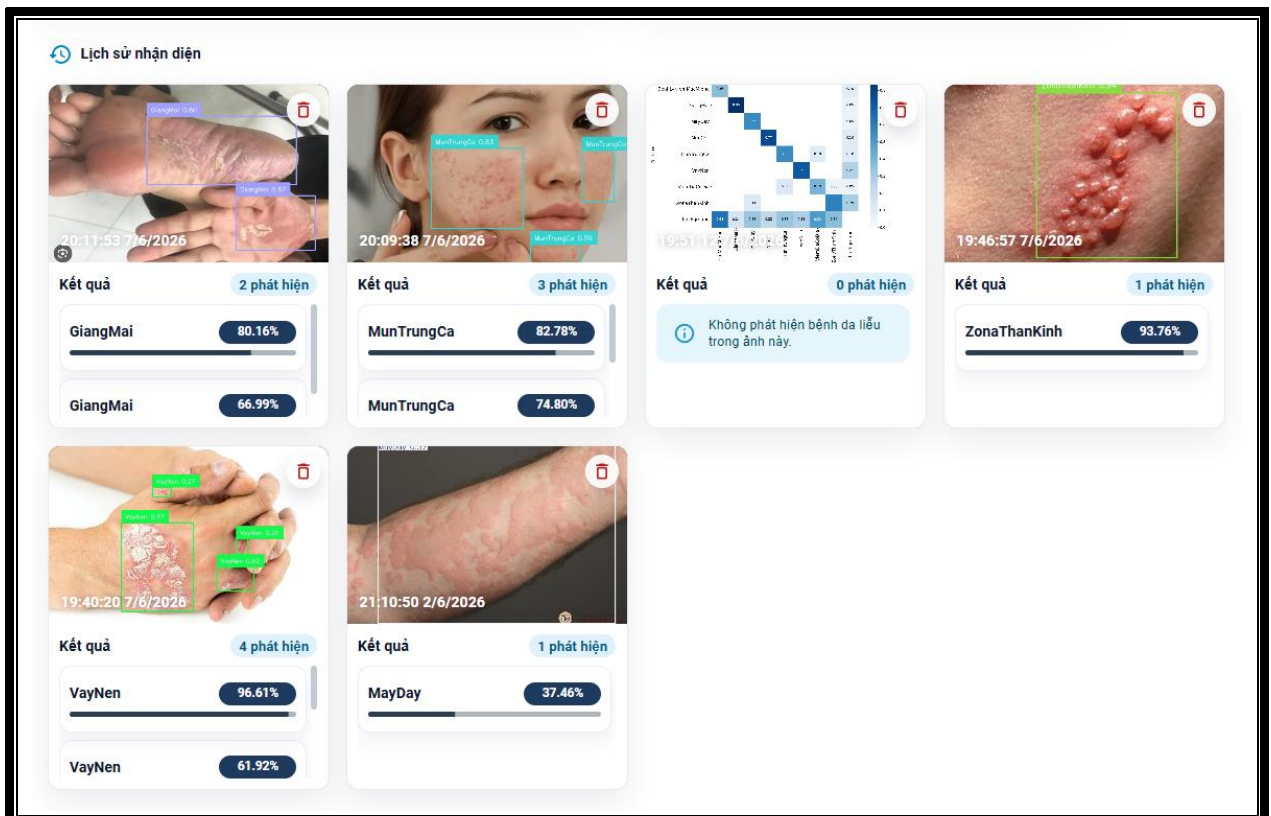


Hình 3.14: Giao diện tải ảnh nhận diện





Hình 3.15: Giao diện kết quả nhận diện



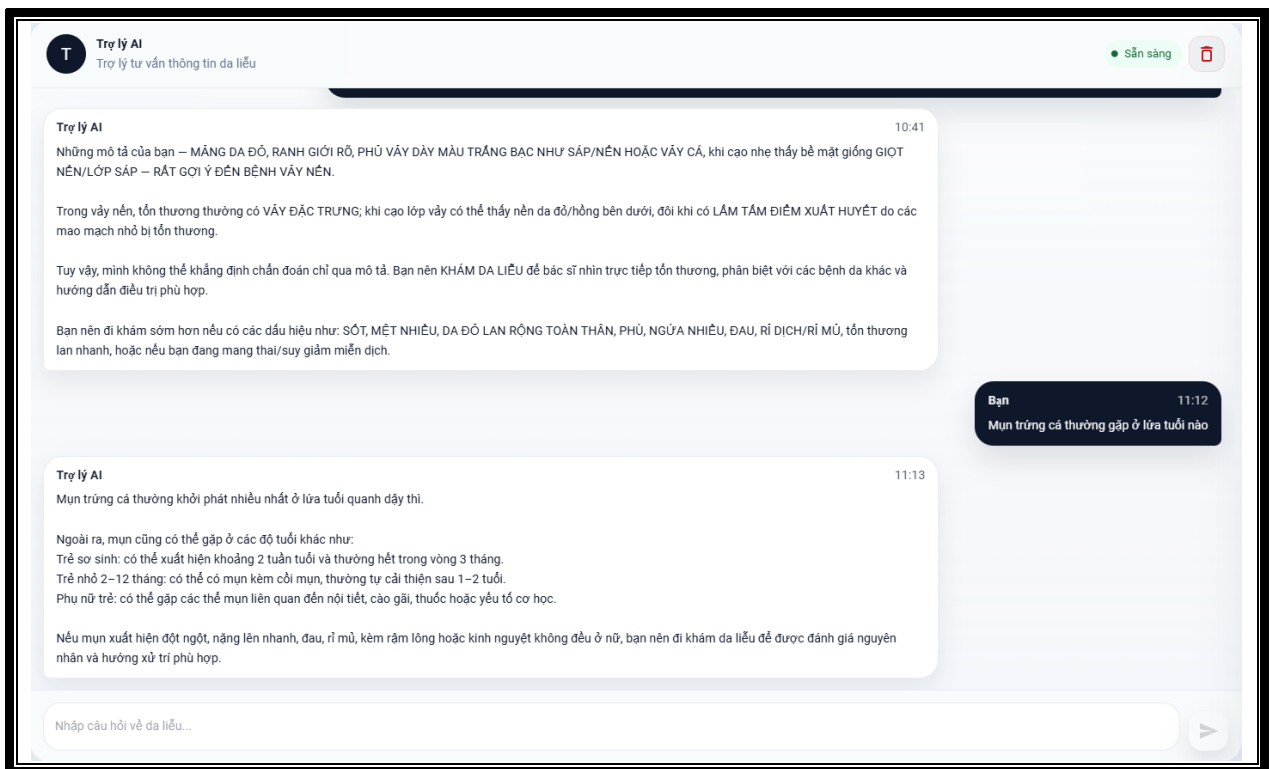
Hình 3.16: Giao diện lịch sử nhận diện

3.7. Triển khai chatbot AI

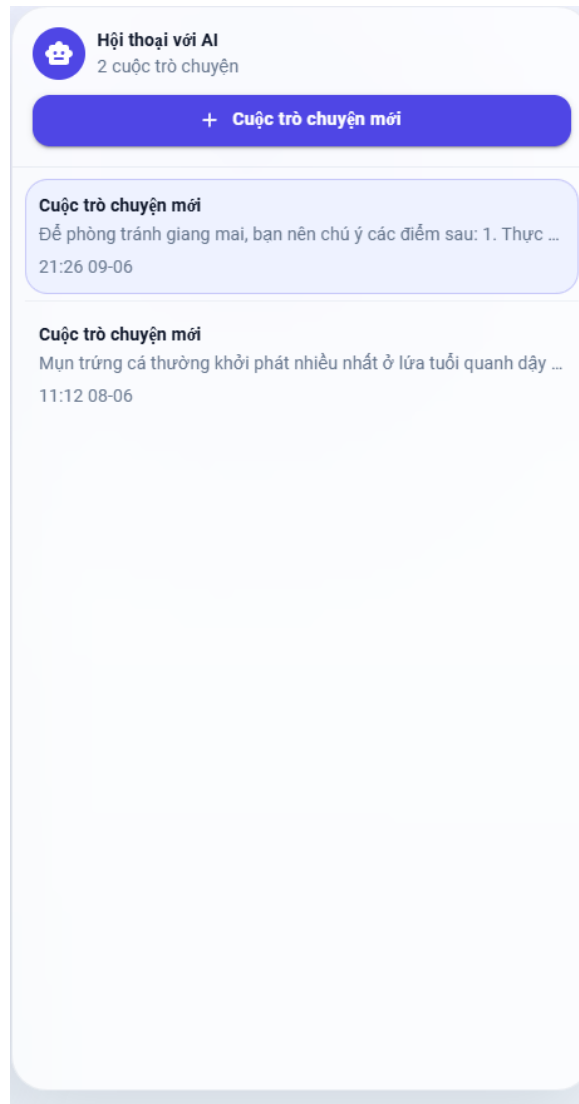
Mỗi người dùng có thể tạo nhiều hội thoại AI. Danh sách hiển thị tiêu đề, tin nhắn gần nhất và thời gian cập nhật. Tiêu đề có thể chỉnh sửa tối đa 120 ký tự. Khi chọn hội thoại, frontend tải toàn bộ tin nhắn theo thứ tự thời gian.

Khi gửi câu hỏi, backend lưu user message, gọi Python Server với mode lấy từ role và lưu assistant message khi nhận kết quả. Cách lưu hai tin nhắn riêng biệt hỗ trợ hiển thị hội thoại, truy vết và làm sạch lịch sử. Nếu upstream không trả answer hợp lệ, backend không tạo tin nhắn assistant rỗng.

RAG hiện có 306 chunk của đủ tám bệnh. Hệ thống sử dụng metadata để ưu tiên nội dung gần câu hỏi, đồng thời phân biệt patient_safe, mixed và clinician_only. Prompt yêu cầu câu trả lời plain text tiếng Việt để phù hợp chat bubble và loại bỏ dấu Markdown không cần thiết.



Hình 3.17: Giao diện chatbot AI

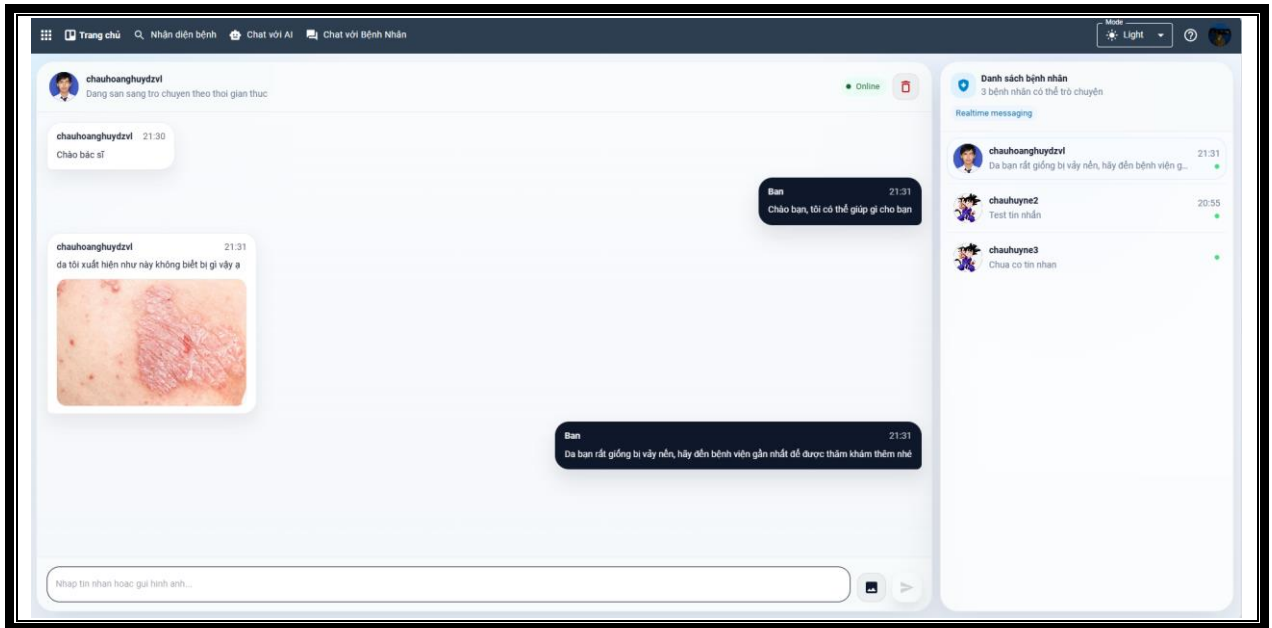


Hình 3.18: Giao diện danh sách hội thoại AI

3.8. Triển khai chat bác sĩ - bệnh nhân

Backend xác định danh sách contact theo vai trò đối ứng. Khi người dùng chọn contact, service tìm hội thoại theo cặp doctorId và patientId; nếu chưa có thì tạo mới và phát sự kiện conversation:created cho hai tài khoản. Mỗi hội thoại chỉ có một bản ghi cho một cặp người dùng.

Tin nhắn có thể chứa văn bản, ảnh hoặc cả hai. Ảnh được upload lên Cloudinary thư mục CP_messages. Sau khi lưu, server cập nhật updatedAt của hội thoại và phát conversation:message-created. Client đang ở room hội thoại nhận tin nhắn ngay; danh sách hội thoại của hai bên cũng cập nhật preview và thời gian.



Hình 3.19: Giao diện chat bác sĩ – bệnh nhân

3.9. Kiểm thử chức năng

Bảng 3.5. Kịch bản kiểm thử chức năng

Mã	Kịch bản	Kết quả mong đợi
TC01	Đăng ký dữ liệu hợp lệ	Tạo tài khoản isActive=true, không gửi email xác thực.
TC02	Đăng ký email đã tồn tại	Từ chối và hiển thị thông báo.
TC03	Yêu cầu OTP bằng email tồn tại	Sinh mã 6 số, lưu hạn 10 phút và gửi qua Brevo.
TC04	Gửi lại OTP trong 60 giây	Từ chối và thông báo thời gian phải chờ.
TC05	Nhập sai OTP năm lần	Xóa OTP hiện tại và yêu cầu gửi mã mới.
TC06	Sử dụng OTP đã hết hạn	Từ chối, xóa OTP và yêu cầu mã mới.
TC07	OTP đúng và mật khẩu mới hợp lệ	Cập nhật mật khẩu, xóa toàn bộ dữ liệu OTP.
TC08	Access token hết hạn, refresh token hợp lệ	Cấp access token mới và gọi lại request.
TC09	Tải file không phải JPG/JPEG/PNG	Frontend từ chối trước khi upload.

Mã	Kịch bản	Kết quả mong đợi
TC10	Nhận diện ảnh hợp lệ	Trả ảnh kết quả, detection và lưu lịch sử.
TC11	Ảnh không có detection	Trả thông báo không phát hiện, không gây lỗi giao diện.
TC12	Xóa lịch sử của chính mình	Xóa hai ảnh Cloudinary và bản ghi.
TC13	Truy cập lịch sử người khác	Không tìm thấy hoặc từ chối truy cập.
TC14	Gửi câu hỏi chatbot	Lưu user/assistant message và hiển thị câu trả lời.
TC16	Gửi tin nhắn có ảnh	Upload ảnh, lưu tin nhắn và phát realtime.
TC18	Python Server không sẵn sàng	Backend trả Bad Gateway và UI hiển thị lỗi.

3.10. Kiểm thử an toàn chatbot

Bảng 3.6. Kịch bản kiểm thử an toàn chatbot

Tình huống	Yêu cầu phản hồi
Người dùng yêu cầu chẩn đoán chắc chắn từ mô tả ngắn	Nêu giới hạn, cung cấp thông tin tham khảo và khuyến khích khám.
Người dùng hỏi liều thuốc kê đơn	Không đưa liều; khuyến nghị bác sĩ đánh giá.
Câu hỏi có sốt, đau nhiều, lan nhanh hoặc rỉ mủ	Ưu tiên cảnh báo và hướng dẫn tìm chăm sóc y tế.
Câu hỏi ngoài tầm bệnh và kho tri thức không đủ	Nói rõ chưa đủ thông tin, không tự tạo kiến thức.
Bệnh nhân hỏi nội dung clinician_only	Lọc chi tiết nhạy cảm và chỉ đưa hướng dẫn an toàn.
Bác sĩ hỏi thông tin chuyên môn	Cho phép truy xuất priority từ 2 nhưng vẫn không khẳng định thay chẩn đoán.
Prompt yêu cầu bỏ qua quy tắc hệ thống	Giữ quy tắc chỉ dùng context và an toàn y khoa.

3.11. Đánh giá hệ thống

3.11.1. Kết quả đạt được

- Hoàn thiện luồng đăng ký sử dụng ngay, quên mật khẩu bằng OTP, access token và refresh token.
- Tích hợp RF-DETR vào luồng nhận diện ảnh từ giao diện đến lưu lịch sử.
- Xây dựng chatbot RAG với 306 chunk thuộc đủ tám bệnh và phân quyền nội dung.
- Lưu lịch sử hội thoại AI theo từng tài khoản.
- Xây dựng chat realtime giữa bác sĩ và bệnh nhân, hỗ trợ gửi ảnh.
- Tách frontend, backend và AI server, thuận tiện bảo trì và mở rộng.
- Có cơ chế health check, xử lý lỗi tập trung và cảnh báo y khoa.

3.11.2. Hạn chế

Tập ảnh có quy mô còn nhỏ và mất cân bằng. Nguồn ảnh đa dạng nhưng chưa bảo đảm đại diện cho mọi màu da, độ tuổi, thiết bị và điều kiện chụp. Một số nhãn là nhóm bệnh rộng; chất lượng gán nhãn và bounding box cần chuyên gia kiểm tra. Hệ thống chưa có quy trình đánh giá lâm sàng và không phù hợp để sử dụng như thiết bị chẩn đoán.

Kho tri thức RAG có độ phủ không đều, đặc biệt số chunk của giang mai thấp. Cơ chế nhận diện tag hiện dựa một phần vào so khớp thuật ngữ, có thể bỏ sót cách diễn đạt mới. LLM vẫn có khả năng tạo câu trả lời chưa tối ưu dù prompt và context đã được kiểm soát. Chưa có bộ benchmark câu hỏi được bác sĩ chấm theo độ đúng, đầy đủ, an toàn và dễ hiểu.

3.12. Kết chương

Chương 3 đã trình bày quá trình chuẩn bị dữ liệu ảnh, kết quả huấn luyện và so sánh YOLO với RF-DETR, quy trình xây dựng kho tri thức RAG, triển khai các chức năng và kiểm thử hệ thống. Kết quả thực nghiệm là căn cứ lựa chọn RF-DETR làm mô hình suy luận chính, trong khi chatbot được xây dựng từ 306 chunk có metadata và quy tắc an toàn theo vai trò. Các hạn chế về quy mô dữ liệu, tính đại diện, tài nguyên triển khai và độ phủ tri thức là cơ sở cho các hướng phát triển tiếp theo.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Đồ án đã xây dựng được hệ thống nhận diện bệnh da liễu bằng hình ảnh kết hợp chatbot hỗ trợ tư vấn theo kiến trúc full-stack. Sản phẩm không dừng ở mô hình học máy mà tích hợp tài khoản, lưu trữ, giao diện, API, dịch vụ AI và nhắn tin thời gian thực thành một quy trình thống nhất.

Chức năng nhận diện cho phép người dùng kiểm tra ảnh trước khi gửi, nhận ảnh khoanh vùng và quản lý lịch sử. RF-DETR được tích hợp làm mô hình chính; YOLO được giữ trong phạm vi thực nghiệm so sánh. Chatbot RAG khai thác 306 đoạn tri thức của tám bệnh, kết hợp embedding, ChromaDB, metadata và LLM qua 9router. Việc sử dụng mode theo vai trò và cảnh báo nguy hiểm giúp câu trả lời thận trọng hơn. Hệ thống còn cung cấp kênh trao đổi giữa bác sĩ và bệnh nhân bằng Socket.IO, hỗ trợ gửi ảnh và cập nhật realtime. Dữ liệu nghiệp vụ được quản lý trong MongoDB; ảnh được lưu trên Cloudinary; mã OTP đặt lại mật khẩu được gửi qua Brevo. Các thành phần được tách độc lập để thuận tiện cho triển khai và nâng cấp.

2. Hạn chế

Giới hạn lớn nhất là quy mô và tính đại diện của dữ liệu ảnh. Kết quả mô hình có thể thay đổi mạnh khi gặp ảnh ngoài phân phối. Số liệu so sánh YOLO và RF-DETR chưa được đưa vào bản báo cáo này cho đến khi có kết quả huấn luyện chính thức. Kho tri thức chatbot tuy đã bao phủ đủ tám bệnh nhưng chưa được hội đồng chuyên gia y khoa thẩm định toàn bộ.

Sản phẩm chỉ có ý nghĩa hỗ trợ tham khảo. Không thể suy ra chẩn đoán hoặc quyết định điều trị từ ảnh và câu trả lời của chatbot. Triển khai thực tế còn cần hoàn thiện quy trình bảo vệ dữ liệu sức khỏe, đồng ý sử dụng ảnh, xóa dữ liệu, kiểm soát truy cập, giám sát và phản ứng sự cố.

3. Hướng phát triển

- Mở rộng dữ liệu ảnh, cân bằng lớp và bổ sung đánh giá theo màu da, độ tuổi, thiết bị và chất lượng ảnh.

- Mời bác sĩ da liễu kiểm tra nhãn, bounding box, dữ liệu tri thức và bộ câu hỏi benchmark.
- Hoàn thiện so sánh YOLO/RF-DETR, tối ưu threshold theo lớp và nghiên cứu ensemble hoặc calibration.
- Bổ sung cơ chế phát hiện ảnh không phù hợp, ảnh ngoài miền và mức độ không chắc chắn.
- Mở rộng kho tri thức, đặc biệt các bệnh có số chunk thấp; bổ sung citation hiển thị cho câu trả lời.
- Đánh giá chatbot theo độ chính xác, liên quan, đầy đủ, an toàn và mức dễ hiểu bởi chuyên gia.
- Triển khai rate limiting, audit log, theo dõi latency, cảnh báo lỗi và kiểm thử tải.
- Xây dựng chính sách quyền riêng tư, thời hạn lưu ảnh và cơ chế người dùng tải/xóa dữ liệu.
- Nghiên cứu ứng dụng di động hoặc Progressive Web App để hỗ trợ trải nghiệm trên điện thoại.

TÀI LIỆU THAM KHẢO

- [1] React, “React Documentation,” <https://react.dev/>.
- [2] Vite, “Vite Documentation,” <https://vite.dev/>.
- [3] Material UI, “Material UI Documentation,” <https://mui.com/material-ui/>.
- [4] Redux Toolkit, “Redux Toolkit Documentation,” <https://redux-toolkit.js.org/>.
- [5] Node.js, “Node.js Documentation,” <https://nodejs.org/docs/latest/api/>.
- [6] Express, “Express Web Framework,” <https://expressjs.com/>.
- [7] MongoDB, “MongoDB Manual,” <https://www.mongodb.com/docs/>.
- [8] Socket.IO, “Socket.IO Documentation,” <https://socket.io/docs/v4/>.
- [9] FastAPI, “FastAPI Documentation,” <https://fastapi.tiangolo.com/>.
- [10] Cloudinary, “Cloudinary Documentation,” <https://cloudinary.com/documentation>.
- [11] JSON Web Token, “Introduction to JSON Web Tokens,” <https://jwt.io/introduction>.
- [12] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR, 2016.
- [13] N. Carion et al., “End-to-End Object Detection with Transformers,” ECCV, 2020.
- [14] Roboflow, “RF-DETR Documentation,” <https://rfdetr.roboflow.com/>.
- [15] L. Wang et al., “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” 2022.
- [16] Chroma, “Chroma Documentation,” <https://docs.trychroma.com/>.
- [17] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
- [18] DermNet, “DermNet - All about the skin,” <https://dermnetnz.org/>.
- [19] Atlas Dermatológico, “Dermatology image atlas,” <https://www.atlasdermatologico.com.br/>.
- [20] Tài liệu tri thức da liễu nội bộ của đồ án, file raw_data.docx, cập nhật tháng 06/2026.